



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(национальный исследовательский университет)»

---

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806

Группа М8О-401Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

---

## ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему: Система управления молочным животноводством на базе доильных роботов,  
совместимых с Lely Astronaut

|               |                          |         |
|---------------|--------------------------|---------|
| Автор ВКРБ:   | Кабанов Антон Алексеевич | (_____) |
| Руководитель: | Ухов Петр Александрович  | (_____) |
| Консультант:  | —                        | (_____) |
| Консультант:  | —                        | (_____) |
| Рецензент:    | —                        | (_____) |

**К защите допустить**

|                                                  |                         |         |
|--------------------------------------------------|-------------------------|---------|
| Заведующий кафедрой № 806<br>_____ мая 2026 года | Крылов Сергей Сергеевич | (_____) |
|--------------------------------------------------|-------------------------|---------|

Москва 2026

## РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 78 страниц, 24 рисунков, 7 таблиц, 42 использованных источников, 4 приложений.

ИНФОРМАЦИОННАЯ СИСТЕМА, МОЛОЧНАЯ ФЕРМА, RUST, SVELTEKIT, МАШИННОЕ ОБУЧЕНИЕ, ПРЕДИКТИВНАЯ АНАЛИТИКА, ПРОГНОЗИРОВАНИЕ НАДОЕВ, ОБНАРУЖЕНИЕ МАСТИТА, ИНТЕГРАЦИЯ LELY, DOCKER

Объектом исследования является процесс автоматизации управления молочной фермой.

Предметом исследования — методы и средства разработки информационных систем управления молочными фермами с интеграцией роботизированного доильного оборудования и предиктивной аналитикой на основе моделей машинного обучения.

Цель работы — разработка информационной системы, обеспечивающей комплексный учёт животных, надоев молока, воспроизводства, кормления, фитнес-мониторинга, пастьбы и интеграцию с роботизированным доильным оборудованием Lely, а также модуля предиктивной аналитики на основе моделей машинного обучения.

Основное содержание работы состоит в проектировании и реализации трёхзвенной архитектуры: серверной части на языке Rust с фреймворком Axum (25 модулей бизнес-логики, JWT-аутентификация, middleware безопасности, интеграция с Lely Horizon), клиентского веб-приложения на SvelteKit (более 24 страниц, адаптивный интерфейс, более 16 UI-компонентов) и модуля аналитики с 11 моделями машинного обучения на Python/FastAPI (XGBoost, Prophet, KMeans, Isolation Forest). Серверная часть содержит собственные алгоритмы прогнозирования: метод Хольта–Винтерса, кривую лактации Вуда, индекс здоровья на основе Z-оценок и систему сравнения 8 моделей прогнозирования временных рядов с автоматическим выбором оптимальной. База данных PostgreSQL (более 35 таблиц) использует TimescaleDB для эффективной работы с временными рядами.

Основным результатом работы является информационная система управления молочной фермой, покрывающая полный цикл управления стадом: от учёта и мониторинга до предиктивной аналитики с доверительными интервалами и раннего обнаружения заболеваний (мастит,

кетоз, охота).

Данные результаты предназначены для автоматизации процессов учёта и мониторинга на молочных фермах, оснащённых роботизированным доильным оборудованием, повышения эффективности управления стадом и снижения экономических потерь от заболеваний животных.

Внедрение системы позволяет сократить время на ведение документации, повысить точность прогнозирования продуктивности стада и своевременно выявлять заболевания на ранних стадиях, когда клинические признаки ещё не очевидны.

## СОДЕРЖАНИЕ

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ . . . . .                                                     | 6  |
| ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ . . . . .                                         | 8  |
| ВВЕДЕНИЕ . . . . .                                                                  | 10 |
| 1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНИЧЕСКОЕ ЗАДАНИЕ . . . . .                                | 14 |
| 1.1 Точное животноводство и автоматизация управления<br>молочными фермами . . . . . | 14 |
| 1.2 Обзор существующих решений . . . . .                                            | 15 |
| 1.3 Техническое задание . . . . .                                                   | 17 |
| 1.4 Стекло технологий . . . . .                                                     | 19 |
| 2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ . . . . .                                     | 21 |
| 2.1 Архитектура системы . . . . .                                                   | 21 |
| 2.2 Модель данных . . . . .                                                         | 23 |
| 2.3 Серверная часть (Backend) . . . . .                                             | 25 |
| 2.3.1 Маршрутизация и обработка запросов . . . . .                                  | 26 |
| 2.3.2 Аутентификация и авторизация . . . . .                                        | 27 |
| 2.3.3 Промежуточное программное обеспечение (Middleware) . . . . .                  | 28 |
| 2.3.4 Интеграция с Lely . . . . .                                                   | 28 |
| 2.4 Клиентское веб-приложение (Frontend) . . . . .                                  | 29 |
| 2.4.1 Маршрутизация и навигация . . . . .                                           | 30 |
| 2.4.2 Аутентификация на стороне клиента . . . . .                                   | 31 |
| 2.4.3 API-клиент . . . . .                                                          | 31 |
| 2.4.4 Компоненты пользовательского интерфейса . . . . .                             | 31 |
| 2.4.5 Генерация отчётов . . . . .                                                   | 32 |
| 2.5 Модуль машинного обучения (Analytics-ML) . . . . .                              | 33 |
| 2.5.1 Модели машинного обучения . . . . .                                           | 34 |
| 2.5.2 Инженерия признаков . . . . .                                                 | 37 |
| 2.5.3 MLOps: обучение, переобучение и мониторинг . . . . .                          | 38 |
| 2.6 Предиктивная аналитика в серверной части . . . . .                              | 38 |
| 2.6.1 Сравнение моделей прогнозирования временных рядов . . . . .                   | 40 |
| 2.6.2 Ансамблевый прогноз . . . . .                                                 | 42 |
| 2.6.3 Объяснимость прогнозов (SHAP) . . . . .                                       | 43 |
| 2.7 Мониторинг . . . . .                                                            | 44 |
| 2.8 Непрерывная интеграция и доставка (CI/CD) . . . . .                             | 46 |

|       |                                                       |    |
|-------|-------------------------------------------------------|----|
| 2.9   | Развёртывание . . . . .                               | 47 |
| 3     | ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РАБОТЫ СИСТЕМЫ . . .      | 49 |
| 3.1   | Стратегия тестирования . . . . .                      | 49 |
| 3.2   | Тестирование серверной части . . . . .                | 49 |
| 3.3   | Тестирование клиентского приложения . . . . .         | 50 |
| 3.4   | Тестирование модуля машинного обучения . . . . .      | 51 |
| 3.5   | Нагрузочное тестирование . . . . .                    | 51 |
| 3.6   | Демонстрация системы . . . . .                        | 51 |
| 3.6.1 | Аутентификация . . . . .                              | 52 |
| 3.6.2 | Дашборд . . . . .                                     | 53 |
| 3.6.3 | Управление животными . . . . .                        | 54 |
| 3.6.4 | Отчёты и аналитика . . . . .                          | 56 |
| 3.6.5 | Интеграция с Lely . . . . .                           | 57 |
| 3.6.6 | Оповещения . . . . .                                  | 57 |
| 3.7   | Мониторинг работоспособности . . . . .                | 57 |
|       | ЗАКЛЮЧЕНИЕ . . . . .                                  | 59 |
|       | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .            | 62 |
|       | ПРИЛОЖЕНИЕ А Исходный код . . . . .                   | 66 |
|       | ПРИЛОЖЕНИЕ Б Описание таблиц базы данных . . . . .    | 67 |
|       | ПРИЛОЖЕНИЕ В Визуализация модели данных . . . . .     | 70 |
|       | ПРИЛОЖЕНИЕ Г Примеры API-запросов и ответов . . . . . | 74 |

## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Информационная система — совокупность аппаратных и программных компонентов, обеспечивающих сбор, хранение, обработку и выдачу информации для решения задач управления

REST API — архитектурный стиль взаимодействия распределённых компонентов сетевых приложений, основанный на протоколе HTTP и принципах REST

Машинное обучение — раздел искусственного интеллекта, изучающий методы построения моделей, способных обучаться на данных и делать прогнозы или принимать решения без явного программирования

Временной ряд — последовательность данных, собранных или измеренных в последовательные моменты времени через равные или неравные промежутки

Лактация — процесс образования и выделения молока молочной железой сельскохозяйственных животных

Охота (эструс) — период половой активности у самок млекопитающих, во время которого происходит овуляция и возможно осеменение

Мастит — воспалительное заболевание вымени коровы, вызываемое бактериальной инфекцией и приводящее к снижению качества и объёма молока

Кетоз — метаболическое нарушение у коров в ранний послеродовой период, характеризующееся повышенным содержанием кетоновых тел в крови

Выбраковка — процесс исключения животного из стада по производственным, ветеринарным или экономическим причинам

Z-оценка — стандартизованное отклонение значения от среднего:  $z = (x - \mu) / \sigma$ , где  $\mu$  — среднее по стаду,  $\sigma$  — стандартное отклонение.

Значения  $|z| > 2$  считаются статистически аномальными

Индекс здоровья — комплексный показатель состояния животного, рассчитываемый как взвешенная сумма Z-оценок надоя, жвачки, активности и SCC. Принимает значения от 0 до 100, где  $\geq 80$  — низкий риск,  $< 40$  — критический

Доверительный интервал — диапазон значений, в котором с заданной вероятностью находится истинное значение прогнозируемой величины. В

данной работе используются квантильные интервалы  $q_{10}/q_{90}$ , содержащие 80 процентов распределения прогнозов

SHAP — SHapley Additive exPlanations — метод объяснимости машинного обучения, основанный на значениях Шепли из теории кооперативных игр. Показывает вклад каждого признака в прогноз модели

Ансамбль моделей — метод объединения прогнозов нескольких моделей в один итоговый прогноз. В данной работе используется взвешенное среднее с весами, обратно пропорциональными MAPE каждой модели

## ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

ИС — информационная система

БД — база данных

СУБД — система управления базами данных

API — application programming interface

REST — representational state transfer

МО — машинное обучение

CI — continuous integration

CD — continuous delivery

JWT — JSON web token

HTTP — hypertext transfer protocol

HTTPS — HTTP secure

SSL — secure sockets layer

TLS — transport layer security

JSON — JavaScript object notation

CSV — comma-separated values

PDF — portable document format

UI — user interface

SCC — somatic cell count

BCS — body condition score

FPR — fat-protein ratio

DIM — days in milk

SSE — server-sent events

CORS — cross-origin resource sharing

SPA — single-page application

SSR — server-side rendering

GHCR — GitHub container registry

SRE — site reliability engineering

DOM — document object model

MAPE — mean absolute percentage error

RMSE — root mean squared error

SES — simple exponential smoothing



ARIMA — autoregressive integrated moving average

KMeans — k-means clustering

## ВВЕДЕНИЕ

Актуальность темы исследования. Молочное животноводство является одной из ключевых отраслей агропромышленного комплекса: по данным ФАО, мировое производство коровьего молока превышает 750 млн тонн в год и продолжает расти со среднегодовым темпом 1,5–2 процента [1]. Эффективность управления молочной фермой напрямую влияет на рентабельность производства: по данным исследований, своевременное выявление мастита снижает экономические потери на 30–50 процентов от потенциального ущерба, а оптимизация кормления позволяет повысить надои на 5–15 процентов без увеличения затрат [2].

Концепция точного животноводства (precision livestock farming), предложенная Д. Беркмансом в 2008 году, предполагает непрерывный автоматизированный мониторинг состояния животных с использованием датчиков и информационных технологий [1]. Современные молочные фермы оснащаются роботизированными доильными установками (например, Lely Astronaut, DeLaval VMS), датчиками активности и жвачки (Lely Qwes, SCR Heatime), автоматическими кормушками, генерирующими значительные объёмы данных. Одна корова в среднем генерирует от 200 до 500 точек данных в сутки (надои по четвертям вымени, активность, жвачка, температура, потребление корма), что для стада в 500 голов составляет до 250 000 записей в день [2]. Без специализированной информационной системы оперативная обработка и анализ этих данных невозможны, что приводит к запоздалому выявлению заболеваний, неоптимальному кормлению и потерям продукции.

Существующие системы управления молочными фермами (Lely Horizon, DeLaval DelPro, SCR Heatime, 1C: Управление фермой) обладают существенными ограничениями: монолитная архитектура, затрудняющая расширение; отсутствие встроенных средств предиктивной аналитики (за исключением SCR Heatime, ограниченного обнаружением охоты); привязка к оборудованию одного производителя; закрытый API, не позволяющий внедрять собственные модели машинного обучения. При этом задачи прогнозирования надоев, раннего выявления мастита и кетоза, кластеризации стада и оптимизации кормления требуют именно кастомных моделей, обученных на данных конкретной фермы. Поэтому актуальна задача создания

информационной системы, объединяющей полный цикл управления стадом, интеграцию с роботизированным оборудованием и модули машинного обучения в единой открытой платформе.

Объектом исследования является процесс автоматизации управления молочной фермой. Предметом исследования — методы и средства разработки информационных систем управления молочными фермами с интеграцией роботизированного доильного оборудования и предиктивной аналитикой на основе моделей машинного обучения. Цель работы — разработка информационной системы управления молочной фермой с интеграцией роботизированного доильного оборудования Lely и предиктивной аналитикой на основе моделей машинного обучения.

Для достижения поставленной цели были решены следующие задачи. Проведён анализ предметной области и существующих решений, на основании которого сформулировано техническое задание и обоснован выбор стека технологий. Спроектирована трёхзвенная архитектура: серверный API на Rust/Axum, клиентское веб-приложение на SvelteKit и модуль машинного обучения на Python/FastAPI. Спроектирована модель данных базы данных PostgreSQL, содержащая более 35 таблиц, с расширением TimescaleDB для работы с временными рядами. Реализован серверный API, включающий аутентификацию на основе JWT, авторизацию с разграничением ролей, middleware-стек безопасности (CSRF, rate limiting, gzip) и более 25 модулей бизнес-логики. Реализована интеграция с облачным API Lely Horizon, обеспечивающая автоматическую синхронизацию 23 типов данных с шифрованием учётных данных. Разработано клиентское веб-приложение на SvelteKit с более чем 24 страницами, адаптивным интерфейсом и переиспользуемыми UI-компонентами. Реализован модуль машинного обучения с 11 моделями: прогноз надоев (XGBoost с квантильной регрессией, Prophet), обнаружение заболеваний (мастит, кетоз), выявление охоты, кластеризация стада (KMeans), аномалии оборудования (Isolation Forest) и рекомендации по кормлению. Реализованы алгоритмы аналитики в серверной части: метод Хольта–Винтерса для прогнозирования временных рядов, кривая лактации Вуда, индекс здоровья и оптимизатор запуска. Реализован модуль мониторинга на базе стека Grafana LGTM. Разработан конвейер CI/CD на GitHub Actions с параллельным тестированием трёх компонентов и автоматическим деплоем. Проведено функциональное и нагрузочное

тестирование системы.

Методы исследования. В работе использованы методы системного анализа, проектирования реляционных баз данных, машинного обучения (градиентный бустинг, кластеризация, обнаружение аномалий), анализа временных рядов (экспоненциальное сглаживание Хольта–Винтерса, ряды Фурье, ARIMA, линейная регрессия), а также методы программной инженерии (контейнеризация, CI/CD, мониторинг).

Научная новизна заключается в следующем. Предложена архитектура информационной системы, интегрирующая серверные алгоритмы аналитики на Rust (8 моделей прогнозирования временных рядов с автоматическим сравнением по MAPE/RMSE) с внешним модулем машинного обучения на Python, что обеспечивает работу предиктивной аналитики даже при недоступности ML-сервиса. Разработан ансамблевый метод объединения прогнозов ML и Rust-моделей с весами, обратно пропорциональными MAPE, что повышает робастность прогнозирования. Внедрена объяснимость прогнозов на основе метода SHAP с визуализацией вклада признаков в интерфейсе. Разработан комплексный индекс здоровья животного на основе взвешенных Z-оценок надоя, жвачки, активности и SCC с динамической визуализацией тренда за 90 дней. Реализована система сравнения моделей прогнозирования (SES, SMA, WMA, линейная регрессия, метод Хольта, Хольт–Винтерс, ряд Фурье, ARIMA) с автоматическим выбором лучшей модели по минимальному MAPE на тестовой выборке.

Практическая значимость работы заключается в том, что разработанная информационная система может быть использована на молочных фермах, оснащённых роботизированным доильным оборудованием Lely, для автоматизации процессов учёта и мониторинга. Внедрение системы позволяет повысить эффективность управления стадом за счёт предиктивной аналитики, сократить время на ведение документации, своевременно выявлять заболевания животных на ранних стадиях и оптимизировать кормление с учётом индивидуальных потребностей каждой коровы. Модуль машинного обучения обеспечивает прогнозирование надоев с доверительными интервалами ( $q_{10}/q_{90}$ ), что позволяет фермеру количественно оценивать неопределённость прогнозов.

Работа основывается на следующих технологиях и инструментах: Rust [3] — язык программирования серверной части; Axum [4] —

веб-фреймворк; SQLx [5] — асинхронный клиент PostgreSQL [6]; TimescaleDB [7] — расширение для работы с временными рядами; SvelteKit [8] — фреймворк для клиентского приложения; Tailwind CSS [9] и Chart.js [10] — средства стилизации и визуализации; XGBoost [11] и Prophet [12] — библиотеки машинного обучения; FastAPI [13] — фреймворк для модуля аналитики; Docker [14] и Docker Compose [15] — средства контейнеризации; Nginx [16] — обратный прокси; Prometheus [17] и Grafana [18] — система мониторинга; GitHub Actions [19] — платформа CI/CD.

Структура работы. Выпускная квалификационная работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе проведён анализ предметной области: рассмотрена концепция точного животноводства и её применение в молочном животноводстве; выполнен обзор и сравнительный анализ существующих решений; сформулировано техническое задание; обоснован выбор стека технологий. Во второй главе описаны проектирование и разработка системы: архитектура, модель данных базы данных, серверная часть (аутентификация, авторизация, middleware, интеграция с Lely), клиентское веб-приложение, модуль машинного обучения (11 моделей, MLOps), предиктивная аналитика в серверной части (8 моделей прогнозирования, индекс здоровья, кривая лактации Вуда), мониторинг, CI/CD и развёртывание. В третьей главе описаны тестирование и демонстрация работы системы: стратегия тестирования, тесты серверной части, клиентского приложения и модуля машинного обучения, нагрузочное тестирование, демонстрация интерфейса и мониторинг работоспособности. В заключении сформулированы основные результаты работы и перспективы дальнейшего развития системы.

# 1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНИЧЕСКОЕ ЗАДАНИЕ

## 1.1 Точное животноводство и автоматизация управления молочными фермами

Концепция точного животноводства (precision livestock farming, PLF) [1] предполагает непрерывный автоматизированный мониторинг состояния животных с использованием датчиков и информационных технологий для принятия управленческих решений в реальном времени. Данная концепция была предложена Д. Беркмансом и К. Беркманс в 2008 году и с тех пор получила широкое распространение в молочном животноводстве [2]. Согласно исследованию Европейской комиссии по сельскому хозяйству, внедрение PLF-технологий позволяет снизить использование антибиотиков на 20–30 процентов, повысить продуктивность стада на 10–15 процентов и сократить трудозатраты на мониторинг в 3–5 раз.

В молочном животноводстве PLF охватывает несколько ключевых направлений:

- а) мониторинг надоев — объём молока по четвертям вымени, скорость доения (кг/мин), электропроводность (мСм/см) как индикатор мастита, количество соматических клеток (SCC, тыс./мл). Типичная корова при доильном роботе Lely Astronaut генерирует 2–3 визита в сутки, каждый из которых содержит 15–20 параметров;
- б) мониторинг здоровья — активность (шаги за час), жвачка (минуты жевания за период), температура тела и индекс потребления корма. Датчики активности (Lely Qwes, SCR HR) фиксируют данные с дискретностью 1–2 часа, что позволяет обнаруживать отклонения в течение суток;
- в) управление воспроизводством — обнаружение охоты по активности (повышение в 3–5 раз относительно базовой линии), контроль стельности и планирование отёлов и запусков. Интервал между отёлами — ключевой KPI: целевое значение 365–400 дней, каждый лишний день обходится в 3–5 долларов потерь от непродуктивного содержания;
- г) управление кормлением — контроль рационов (сухое вещество, netto-energy-lactation), потребление кормов (кг/день) и остатки;

эффективность корма (л молока / кг сухого вещества) является одним из главных экономических показателей;

- д) управление оборудованием — отслеживание состояния доильных роботов, аномалий работы и прогнозирование технического обслуживания.

Роботизированные доильные установки, такие как Lely Astronaut, автоматически собирают данные о каждом доении. Датчики активности (Lely Qwes) фиксируют шаги, жвачку и время приёма корма. Эти данные передаются в облачный сервис Lely Horizon через REST API. Совокупный объём данных для стада из 500 коров составляет до 250 000 записей в сутки, что делает невозможной их ручную обработку.

Раннее выявление таких заболеваний, как мастит и кетоз, критически важно для экономической эффективности. По данным международного исследования [20], средняя стоимость случая клинического мастита составляет от 100 до 300 долларов США с учётом снижения надоев (70 процентов потерь), затрат на лечение (15 процентов) и преждевременной выбраковки (15 процентов). Кетоз поражает до 40 процентов коров в ранний послеродовой период и при невыявлении приводит к потере до 500 кг молока за лактацию. Модели машинного обучения способны анализировать паттерны в многомерных временных рядах и обнаруживать отклонения на ранних стадиях, когда клинические признаки ещё не очевидны [20].

## **1.2 Обзор существующих решений**

На рынке представлены несколько информационных систем управления молочными фермами. Сравнительный анализ приведён в таблице 1.

Таблица 1 – Сравнительный анализ существующих решений

| Критерий              | Lely Horizon | DeLaval DelPro | SCR Heatime | 1С: Управление фермой |
|-----------------------|--------------|----------------|-------------|-----------------------|
| Учёт животных         | Да           | Да             | Нет         | Да                    |
| Учёт надоев           | Да           | Да             | Нет         | Да                    |
| Воспроизводство       | Да           | Да             | Да          | Да                    |
| Кормление             | Да           | Да             | Нет         | Ограничено            |
| Интеграция с роботами | Lely         | DeLaval        | Нет         | Нет                   |
| Модели ML             | Нет          | Нет            | Да (охота)  | Нет                   |
| Открытый API          | Ограничен    | Нет            | Нет         | Нет                   |
| Кастомизация          | Низкая       | Низкая         | Низкая      | Средняя               |
| Стоимость             | Высокая      | Высокая        | Средняя     | Низкая                |

**Lely Horizon** — облачная платформа от производителя доильных роботов Lely. Обеспечивает мониторинг стада и оборудования в реальном времени: отображение текущего надоя, активности, жвачки и состояния здоровья по каждой корове. Интерфейс поддерживает настройку пороговых оповещений (например, падение надоя > 20 процентов за сутки). Однако система ограничена экосистемой Lely: не поддерживает оборудование других производителей, не предоставляет API для интеграции сторонних моделей аналитики и не позволяет добавлять собственные правила оповещений. Для ферм, использующих смешанное оборудование, это является критическим ограничением.

**DeLaval DelPro** — система управления фермой от DeLaval. Предлагает учёт животных и надоев, управление доильным залом и мониторинг оборудования. Является проприетарной и привязана к оборудованию DeLaval: данные доильных роботов других производителей недоступны. Не поддерживает интеграцию с оборудованием сторонних производителей, что затрудняет внедрение на фермах с гетерогенным оборудованием. Предиктивная аналитика ограничена базовыми пороговыми оповещениями.

**SCR Heatime** — система мониторинга активности и обнаружения охоты



от SCR Engineers (ныне Allflex). Система специализируется на воспроизводстве: использует данные о активности (шаги) и жвачке для обнаружения охоты с помощью запатентованного алгоритма. Встроенная модель машинного обучения обеспечивает чувствительность обнаружения охоты до 95 процентов при специфичности 90 процентов [1]. Однако SCR Heatime не охватывает полный цикл управления фермой: нет учёта кормления, надоев, запасов и оборудования. Система является закрытой и не предоставляет API.

**1С: Управление фермой** — отечественное решение на платформе 1С:Предприятие. Обеспечивает базовый учёт животных, надоев, воспроизводства и кормления. Преимуществом является локализация, соответствие российскому законодательству и низкая стоимость. Однако система не поддерживает интеграцию с роботизированным оборудованием, не имеет модулей машинного обучения, ограничена в кастомизации отчётов и использует монолитную архитектуру, затрудняющую масштабирование.

Анализ показывает, что ни одно из существующих решений не обеспечивает одновременно:

- а) полный цикл управления стадом от учёта до аналитики;
- б) интеграцию с оборудованием сторонних производителей через открытое API;
- в) встроенные модели машинного обучения для предиктивной аналитики;
- г) открытую расширяемую архитектуру для добавления собственных моделей.

### **1.3 Техническое задание**

На основании проведённого анализа сформулируем техническое задание на разработку информационной системы.

Учёт животных должен обеспечивать ведение реестра с полной биографической информацией (инвентарный номер, кличка, пол, дата рождения, порода, происхождение), историю переводов между группами и местоположениями, деактивацию без удаления записей для сохранения истории, поиск по кличке и инвентарному номеру с поддержкой нечёткого поиска (триграммное сходство `pg_trgm`), импорт из CSV и экспорт карточки в PDF, а также пакетные операции для массовой деактивации и обновления до

500 животных.

Учёт надоев молока должен включать суточные надои (объём в кг, жир, белок, лактозу, SCC, FPR), визиты на доение (время начала и окончания, объём по четвертям вымени, скорость доения, данные робота), результаты лабораторных анализов качества молока и мониторинг резервуара-охладителя.

Управление воспроизводством должно поддерживать регистрацию отёлов с записью данных о телёнке, осеменений с указанием быка-производителя, контроль стельности, регистрацию охот на основе данных активности и управление запусками (сухостойный период).

Управление кормлением должно обеспечивать работу с типами кормов и группами кормов, дневными объёмами (плановое и фактическое потребление) и визитами на кормление (время, объём, остатки).

Фитнес-мониторинг и пастьба должны охватывать активность (шаги за период), жвачку (минуты жевания за период) и данные о пастьбе.

Отчёты и аналитика должны включать генерацию отчётов более 17 типов с экспортом в CSV и PDF, ключевые показатели эффективности (KPI), тренды надоев с прогнозом и прогноз стельных коров.

Интеграция с роботизированным оборудованием должна обеспечивать автоматическую синхронизацию с облачным API Lely Horizon с настраиваемым интервалом для 23 типов данных и шифрованием учётных данных (AES-256-GCM).

Предиктивная аналитика должна включать:

- а) прогноз надоев на корову (с доверительными интервалами q10/q90) и на стадо;
- б) обнаружение мастита и кетоза;
- в) выявление охоты;
- г) оценку риска выбраковки;
- д) кластеризацию стада (KMeans);
- е) обнаружение аномалий оборудования (Isolation Forest);
- ж) рекомендации по кормлению;
- и) индекс здоровья на основе Z-оценок;
- к) сравнение моделей прогнозирования (8 моделей с автоматическим выбором).

Нефункциональные требования включают:

- а) аутентификацию и авторизацию с разграничением ролей (admin/user);
- б) адаптивный веб-интерфейс для настольных и мобильных устройств;
- в) контейнеризацию всех компонентов (Docker);
- г) мониторинг работоспособности (Prometheus, Grafana, Loki, Tempo);
- д) автоматизированное развёртывание через CI/CD (GitHub Actions).

#### 1.4 Стек технологий

Выбор технологий обоснован требованиями к производительности, безопасности и удобству разработки.

Для серверной части выбран язык программирования Rust [3], обеспечивающий безопасность работы с памятью без сборщика мусора за счёт системы владения (ownership) и заимствования (borrowing), что исключает целый класс уязвимостей: использование после освобождения (use-after-free), двойное освобождение (double-free), выход за границы массива. По результатам бенчмарков TechEmpower Framework Benchmarks, веб-фреймворк Actix-web (аналогичного уровня с Axum) демонстрирует пропускную способность более 300 000 запросов/сек на одном ядре, что в 3–5 раз выше Node.js/Express и сопоставимо с C++. Веб-фреймворк Axum [4] построен на асинхронном рантайме Tokio [21] и библиотеке Tower [22], что обеспечивает высокую пропускную способность при обработке HTTP-запросов за счёт неблокирующего ввода-вывода. Библиотека SQLx [5] выполняет проверку SQL-запросов при компиляции (compile-time query checking), исключая ошибки в SQL-коде на этапе сборки.

В качестве основной СУБД выбран PostgreSQL [6] — зрелая (более 25 лет разработки), надёжная (ACID-транзакции) реляционная база данных с расширенной поддержкой типов данных, индексов и хранимых процедур. Расширение TimescaleDB [7] обеспечивает эффективное хранение и агрегацию временных рядов с использованием гипертаблиц [23] и непрерывных агрегатов, что критично для таблиц с суточными надоями, активностью и жвачкой (миллионы записей за год).

Для клиентского приложения выбран фреймворк SvelteKit [8]. В отличие от React и Angular, Svelte 5 [24] не использует виртуальный DOM, а компилирует компоненты в оптимизированный императивный JavaScript-код: размер бандла в 2–3 раза меньше React-аналога, а время рендеринга — в 1,5–2

раза быстрее. SvelteKit поддерживает серверный рендеринг (SSR), улучшающий скорость начальной загрузки. Tailwind CSS [9] используется для стилизации (utility-first подход с автоматическим удалением неиспользуемых классов), Chart.js [10] — для визуализации данных.

Для модуля машинного обучения выбран Python как основной язык в области data science с наибольшей экосистемой библиотек. FastAPI [13] обеспечивает высокопроизводительный API с автоматической генерацией документации (OpenAPI/Swagger) и валидацией типов через Pydantic. XGBoost [11] используется для задач классификации и регрессии благодаря высокой производительности (в 10–20 раз быстрее scikit-learn на тех же данных) и поддержке квантильной регрессии. Prophet [12] — для прогнозирования на уровне стада с автоматической декомпозицией тренда и сезонности. Optuna [25] — для байесовской оптимизации гиперпараметров (TPE-сэмплер).

Инфраструктура базируется на Docker [14] и Docker Compose [15], обеспечивающих контейнеризацию и оркестрацию с изолированными средами для каждого компонента. Nginx [16] выступает в роли обратного прокси с терминацией SSL, rate limiting и кэшированием статики. стек мониторинга Grafana LGTM (Prometheus [17], Grafana [18], Loki [26], Tempo [27]) обеспечивает комплексное наблюдение: метрики, логи и распределённые трассировки. GitHub Actions [19] автоматизирует CI/CD: параллельное тестирование трёх компонентов, сборку Docker-образов и деплой.

## 2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

### 2.1 Архитектура системы

Разработанная информационная система имеет трёхзвенную архитектуру, состоящую из серверной части (backend), клиентского веб-приложения (frontend) и модуля машинного обучения (analytics-ml). Компоненты развёрнуты в контейнерах Docker и объединены через обратный прокси Nginx.

На рисунке 1 представлена архитектура системы. Nginx является единой точкой входа, принимающей HTTP-запросы от клиентов. Запросы к статическим ресурсам и страницам маршрутизируются к контейнеру frontend (SvelteKit), запросы к API — к контейнеру backend (Rust/Axum). Backend обращается к PostgreSQL для хранения данных, к Redis для кэширования и rate limiting, и к analytics-ml для получения прогнозов машинного обучения. Синхронизация с облачным API Lely выполняется периодически по таймеру внутри backend.

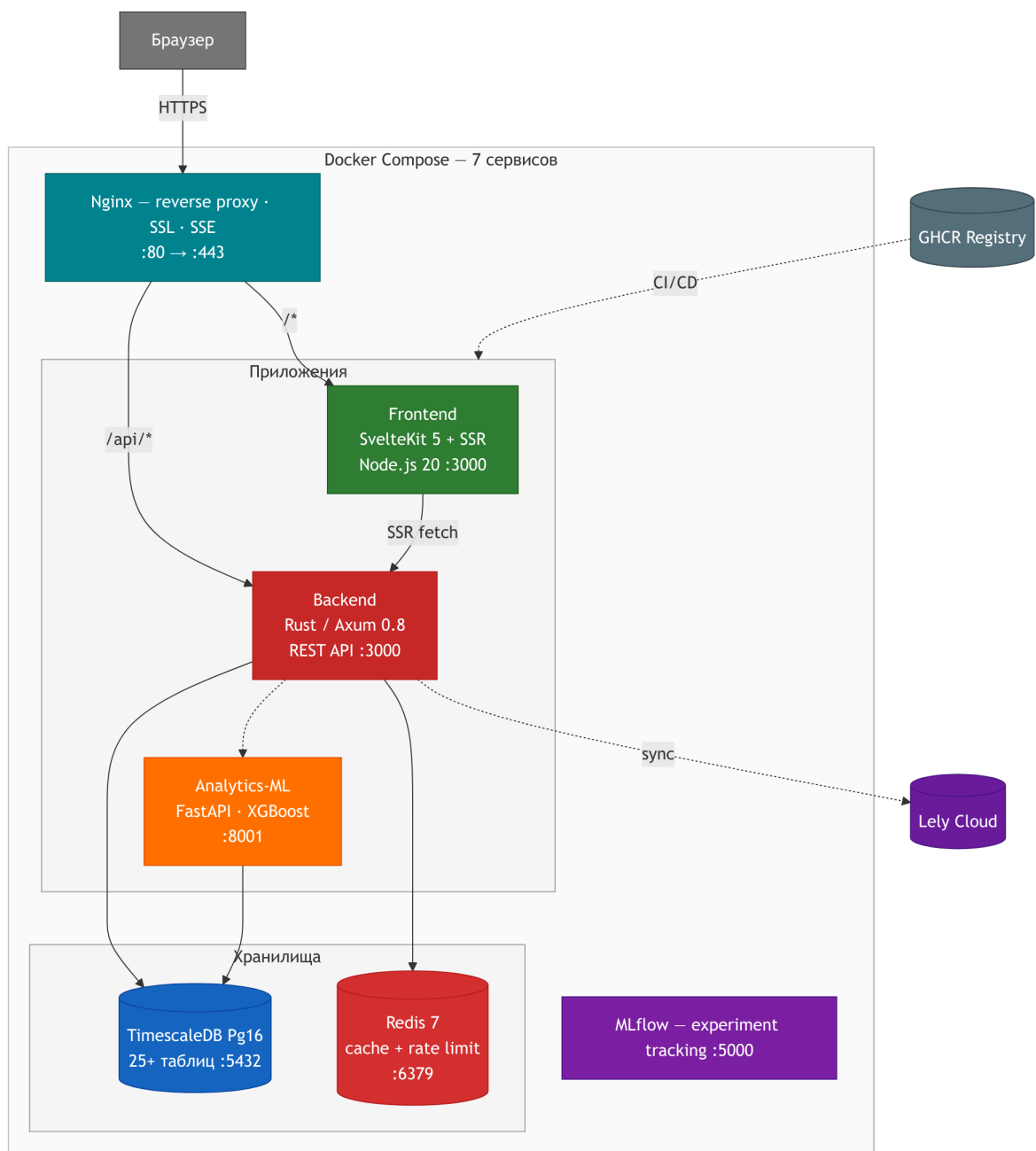


Рисунок 1 – Архитектура информационной системы

Архитектурные решения обоснованы несколькими соображениями. Разделение ответственности обеспечивает чёткое разграничение: backend отвечает за бизнес-логику, frontend — за пользовательский интерфейс, analytics-ml — за модели машинного обучения. Независимое масштабирование позволяет запускать несколько экземпляров backend за балансировщиком нагрузки без изменения остальных компонентов. Технологическая гетерогенность даёт возможность использовать наиболее подходящий стек для каждого компонента (Rust для серверной части, Svelte для клиента, Python для ML). Изоляция сбоев, обеспечиваемая контейнерной

изоляция Docker, гарантирует, что сбой одного компонента не приводит к отказу всей системы.

## 2.2 Модель данных

База данных PostgreSQL содержит более 35 таблиц, обеспечивающих хранение всех сущностей предметной области. Перечень таблиц с назначением приведён в приложении , полная ER-модель — в приложении . На рисунке 2 представлена модель данных системы.

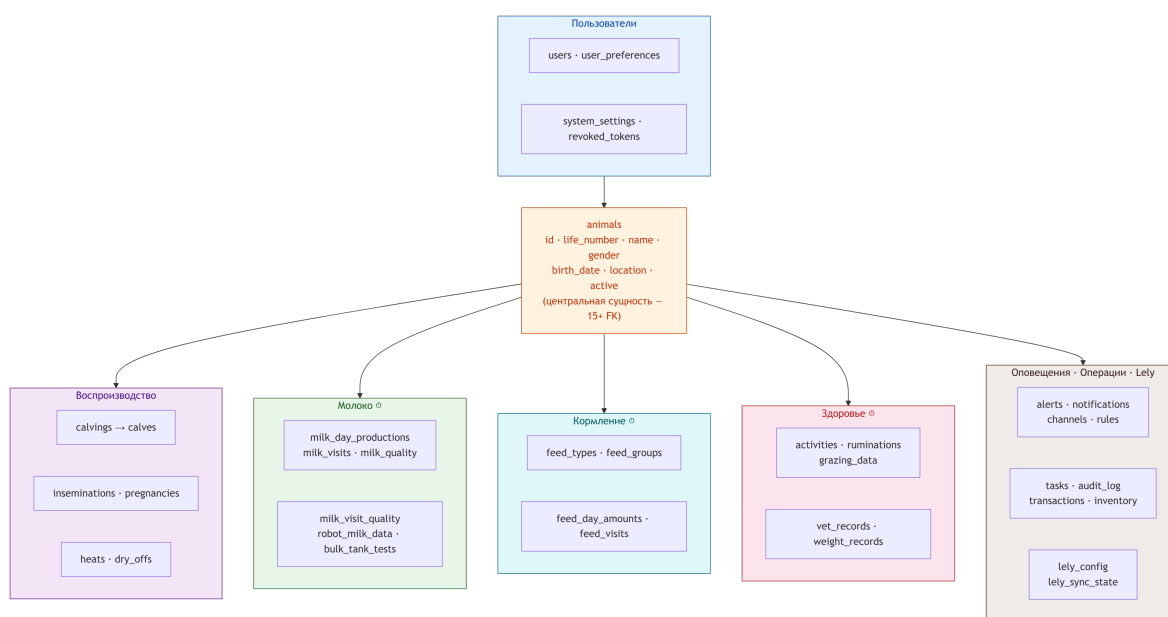


Рисунок 2 – Модель данных информационной системы

Основные таблицы базы данных и их назначение:

**users** — пользователи системы. Содержит имя пользователя, хеш пароля (bcrypt), роль (admin/user), флаг обязательной смены пароля и дату создания. Пароли хранятся в виде хешей с использованием алгоритма bcrypt с автоматической генерацией соли, что исключает утечку паролей при компрометации базы данных. Хеш-функция bcrypt является адаптивной: параметр cost (по умолчанию 12) определяет вычислительную сложность и может быть увеличен по мере роста производительности оборудования.

**animals** — реестр животных. Содержит уникальный идентификатор, инвентарный номер (life\_number), кличку, пол, дату рождения, породу, местоположение, статус активности и множество других полей. Поддерживается флаг active для деактивации животных без удаления записей, что обеспечивает сохранение истории. Инвентарный номер имеет

уникальное ограничение для предотвращения дублирования.

**milk\_day productions** — суточные надои. Привязаны к животному и дате. Содержат объём молока (кг), жир (%), белок (%), лактозу (%), количество соматических клеток (SCC), энергию (MJ) и жирowo-белковое соотношение (FPR). Композитный индекс (animal\_id, date) обеспечивает быстрый поиск надоев за период.

**milk\_visits** — визиты на доение. Содержат информацию о каждом доении: время начала и окончания, объём молока, скорость доения (кг/мин), данные по четвертям вымени (объём, время, процент выдаивания), флаг успешности доения и идентификатор робота.

**milk\_quality** — результаты лабораторных анализов молока. Включает количество соматических клеток (SCC), жир, белок, лактозу, мочевины, бактерии.

**calvings, calves, inseminations, pregnancies, heats, dry\_offs** — данные о воспроизводстве. Каждая таблица связана с животным через внешний ключ. Поддерживается полная история всех событий воспроизводства, включая дату события, результат и примечания.

**feed\_types, feed\_groups, feed\_day\_amounts, feed\_visits** — данные о кормлении. Типы кормов описывают рационы с указанием стоимости и единиц измерения, группы — привязку кормов к группам животных, дневные объёмы — плановое и фактическое потребление, визиты — отдельные посещения кормостанции с временем и объёмом.

**activities, ruminations, grazing\_data** — данные фитнес-мониторинга и пастбы. Активность измеряется в шагах за период, жвачка — в минутах жевания за период. Данные поступают от датчиков или через интеграцию с Lely.

**alerts** — система оповещений. Оповещения создаются автоматически на основе настраиваемых пороговых значений. Каждое оповещение содержит категорию (milk, health, reproduction, feed, equipment), степень важности (high, medium, low), статус (active, acknowledged, resolved), идентификатор животного и текстовое описание.

**lely\_sync\_state** — состояние синхронизации с Lely. Хранит дату последней успешной синхронизации для каждого типа данных, что позволяет инкрементально обновлять информацию и минимизировать объём данных при каждой синхронизации.



**lely\_config** — конфигурация интеграции с Lely. Хранит URL API, зашифрованные логин и пароль, ключ фермы и настройки синхронизации. Учётные данные шифруются с использованием AES-256-GCM [28].

Для эффективной работы с временными рядами (надои, активность, жвачка) используется расширение TimescaleDB [7]. Таблицы с временными рядами преобразованы в гипертаблицы с автоматической партицировкой по времени (по умолчанию — 7 дней на чанк), что ускоряет запросы за определённый период, так как сканируются только релевантные чанки. Непрерывные агрегаты (continuous aggregates) обеспечивают предвычисление суточных и часовых агрегатов, обновляемых в фоновом режиме.

Индексация включает уникальные ограничения (инвентарный номер животного), композитные индексы (животное + дата) и частичные индексы (только активные животные). Расширение pg\_trgm [29] обеспечивает полнотекстовый поиск по кличкам животных с использованием триграммного сходства.

### **2.3 Серверная часть (Backend)**

Серверная часть реализована на языке Rust с использованием фреймворка Axum. Архитектура следует паттерну разделения на слои: HTTP-обработчики (handlers), сервисы бизнес-логики (services), модели данных (models) и промежуточное программное обеспечение (middleware). На рисунке 3 представлена архитектура серверной части.

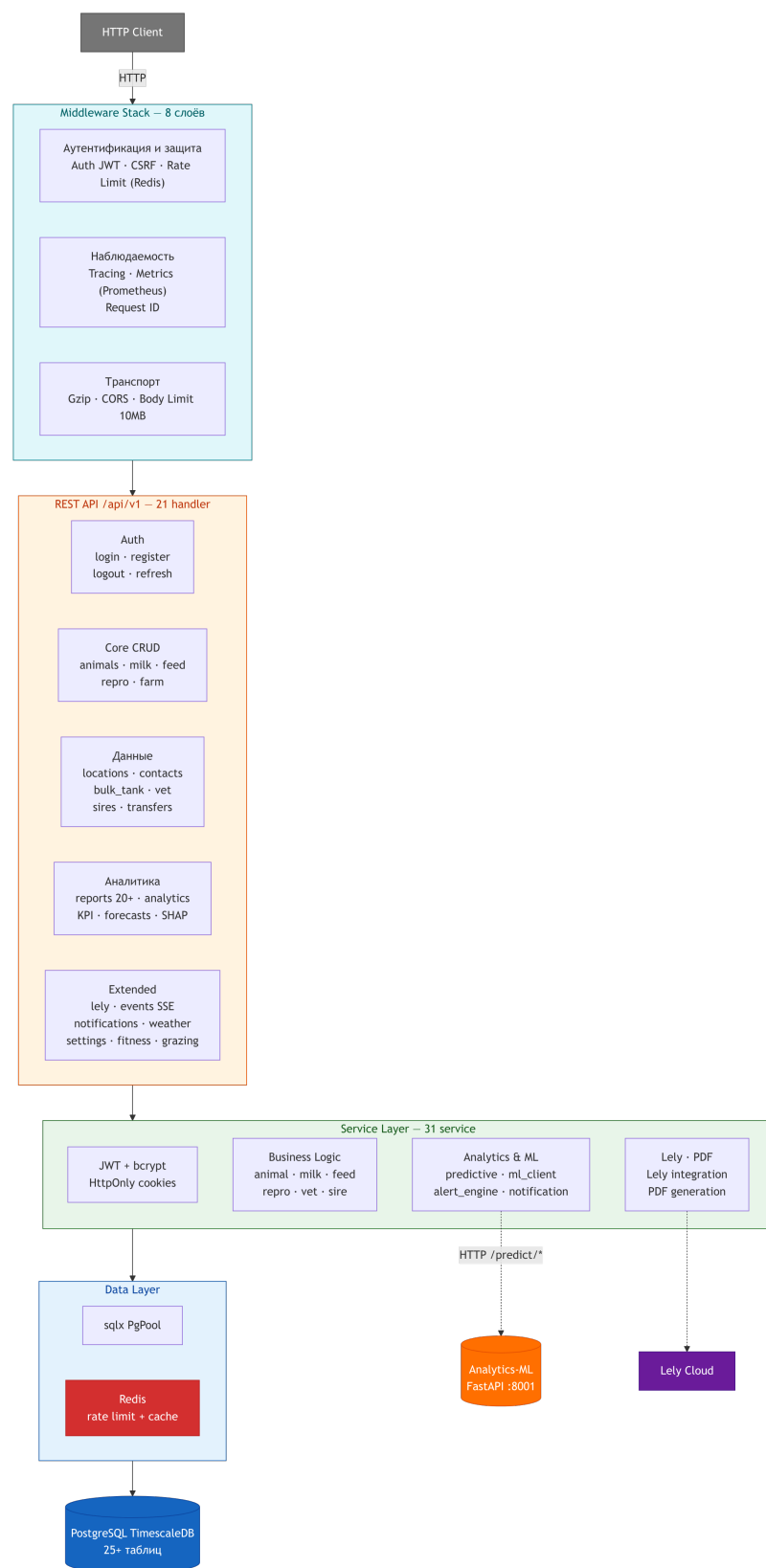


Рисунок 3 – Архитектура серверной части

### 2.3.1 Маршрутизация и обработка запросов

Все API-эндпоинты доступны по префиксу `/api/v1/`. Маршруты

сгруппированы по доменным областям: животные, надои, воспроизводство, кормление, фитнес, отчёты, аналитика, настройки и другие. Общее количество обработчиков — более 25 модулей. Примеры запросов и ответов приведены в приложении .

Документация API генерируется автоматически с помощью библиотеки `utoipa` [30] на основе аннотаций в коде и доступна через Swagger UI по адресу `/api/v1/docs`. Аннотации включают описание эндпоинтов, типы запросов и ответов, коды ошибок. Документация всегда актуальна, так как генерируется из исходного кода.

### 2.3.2 Аутентификация и авторизация

Аутентификация основана на JWT [31] (JSON Web Token) с алгоритмом HS256. При входе в систему сервер выдаёт два токена: короткоживущий `access token` (по умолчанию 15 минут) для авторизации запросов, содержащий имя пользователя (`sub`), роль (`role`), флаг обязательной смены пароля (`must_change_password`) и уникальный идентификатор токена (`jti`); и долгоживущий `refresh token` (по умолчанию 7 дней) для обновления `access token` без повторного ввода пароля.

Токены хранятся в `HttpOnly, SameSite=Strict cookie`, что предотвращает доступ к ним из JavaScript и обеспечивает защиту от CSRF-атак [32]. `Access token` может передаваться также в заголовке `Authorization: Bearer <token>` для поддержки программных клиентов.

Отзыв токенов реализован через таблицу `revoked_tokens`, хранящую JTI отозванных токенов. При каждом запросе проверяется, не был ли токен отозван. Просроченные записи удаляются ежечасно фоновым заданием. Механизм отмены необходим для реализации безопасного выхода из системы: при выходе оба токена (`access` и `refresh`) добавляются в список отозванных.

Авторизация реализована через три уровня доступа, соответствующих типам Ахун-экстракторов. Уровень **Claims** допускает любого аутентифицированного пользователя с валидным `access token`, при этом запрос блокируется, если требуется сменить пароль. Уровень **ClaimsAllowMustChange** также требует аутентификации, но допускает состояние обязательной смены пароля и используется для эндпоинта смены пароля. Уровень **AdminGuard** ограничивает доступ пользователями с ролью

admin и применяется для управления пользователями, настройками системы и резервным копированием.

Время жизни JWT настраивается через таблицу `system_settings` без перезапуска сервера. Изменение вступает в силу для новых токенов.

### 2.3.3 Промежуточное программное обеспечение (Middleware)

Стек `middleware` обеспечивает безопасность, наблюдаемость и ограничение нагрузки. Компоненты `middleware` применяются в порядке, при котором каждый запрос последовательно проходит через все слои. **CORS** настраивает кросс-доменный доступ [33] с конфигурируемыми источниками, методами и разрешёнными заголовками; в продакшн-конфигурации разрешены только домены приложения. **CSRF-защита** проверяет заголовок `Origin` для изменяющих состояние запросов (`POST`, `PUT`, `DELETE`, `PATCH`) и отклоняет запрос при отсутствии или несовпадении `Origin`. **Rate Limiting** ограничивает частоту запросов по IP-адресу (по умолчанию 100 запросов за 60 секунд) с использованием `Redis` при наличии или в памяти в качестве `fallback`; поддерживается определение реального IP через заголовки `X-Forwarded-For` и `X-Real-IP` при работе за обратным прокси. **Gzip-сжатие** уменьшает объём передаваемых данных. **Логирование** обеспечивает трассировку HTTP-запросов и ответов через `tracing` с поддержкой структурированных логов в JSON-формате. **Метрики Prometheus** включают счётчики `http_requests_total` и гистограммы `http_request_duration_seconds` с нормализацией маршрутов. **Идентификаторы запросов** передаются в заголовке `X-Request-Id` для трассировки между сервисами. Наконец, **ограничение размера тела** (10 МБ) предотвращает атаки с чрезмерной нагрузкой.

### 2.3.4 Интеграция с Lely

Модуль интеграции с `Lely` обеспечивает автоматическую двустороннюю синхронизацию данных с облачным API `Lely Horizon`. Синхронизация выполняется по настраиваемому таймеру (по умолчанию каждые 5 минут).

Архитектура модуля включает HTTP-клиент с аутентификацией по токenu и автоматическим обновлением каждые 58 минут (токен `Lely` действует 60 минут), а также повторными попытками с экспоненциальной

задержкой (2 попытки, базовая задержка 1 секунда). Планировщик синхронизации поддерживает параллельные задачи с мьютексом для предотвращения одновременного запуска и инкрементальную синхронизацию по временным диапазонам (чанками по 7 дней для временных рядов). Маппинг данных использует кэш инвентарных номеров (`HashMap<LifeNumber, AnimalId>`) для разрешения Life Number Lely во внутренний идентификатор животного; кэш обновляется перед каждой синхронизацией. Upsert-логика выполняет добавление или обновление записей по уникальному ключу (например, `animal_id + date` для суточных надоев) через `INSERT ON CONFLICT UPDATE`. Учётные данные (логин и пароль Lely) шифруются алгоритмом AES-256-GCM [28], а ключ шифрования задаётся через переменную окружения `LELY_ENCRYPTION_KEY`.

Синхронизируются 23 типа данных: животные, отёлы, осеменения, стельности, охоты, запуски, суточные надои, визиты на доение, качество молока, данные роботов, кормление, активности, жвачка, пастьба, быки-производители, переводы, кровные линии, типы и группы кормов, контакты, местоположения.

Конфигурация интеграции хранится в таблице `lely_config` и может быть изменена через веб-интерфейс без перезапуска сервера. При изменении конфигурации планировщик синхронизации перезапускается через `CancellationToken`.

## **2.4 Клиентское веб-приложение (Frontend)**

Клиентское приложение реализовано на SvelteKit 2 с Svelte 5 [24] и представляет собой одностраничное приложение (SPA) с серверным рендерингом (SSR). На рисунке 4 представлена архитектура клиентского приложения.

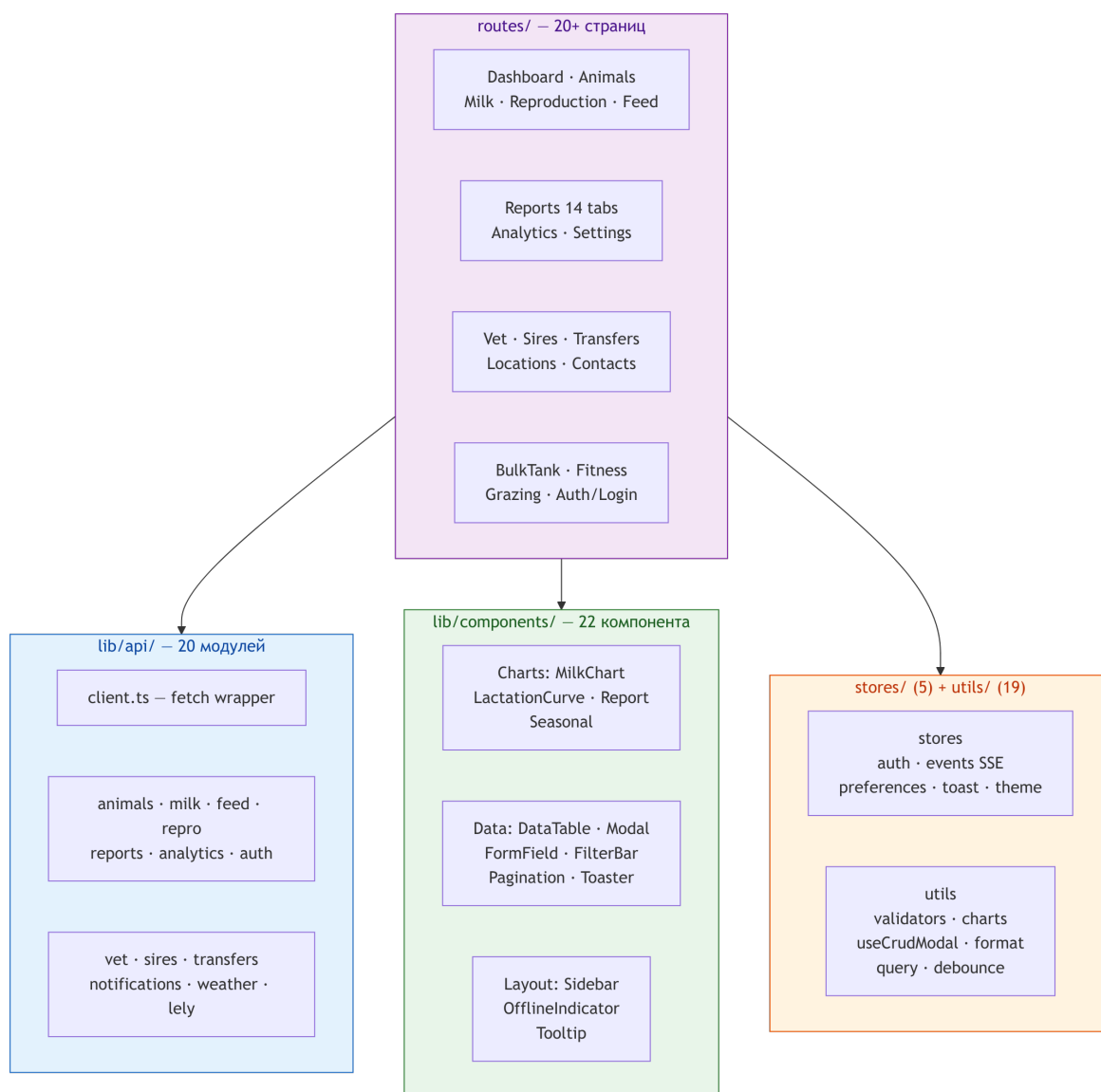


Рисунок 4 – Архитектура клиентского веб-приложения

### 2.4.1 Маршрутизация и навигация

Приложение содержит более 24 страниц, сгруппированных по функциональным областям: дашборд, животные, надой, воспроизводство, кормление, фитнес, пастьба, отчёты, аналитика, настройки, контакты, местоположения, быки, переводы, запасы, оповещения, задачи, ветеринария, финансы, рентабельность, поиск.

Навигация реализована через боковую панель (Sidebar) с иконками из библиотеки Lucide и группировкой разделов. Адаптивная верстка на основе Tailwind CSS обеспечивает корректное отображение на мобильных устройствах: на экранах менее 768px боковая панель сворачивается в гамбургер-меню.

## 2.4.2 Аутентификация на стороне клиента

Серверный хук (`hooks.server.ts`) проверяет наличие JWT в cookie при каждом запросе. Декодируется полезная нагрузка токена (без проверки подписи, только чтение полей) и проверяется срок действия. Неаутентифицированные пользователи перенаправляются на страницу входа, аутентифицированные — не могут перейти на страницу входа.

Управление состоянием аутентификации реализовано через Svelte-стор (`auth store`), который инициализируется при загрузке приложения через `endpoint refresh`. Стор содержит имя пользователя, роль, флаг обязательной смены пароля и состояние аутентификации.

## 2.4.3 API-клиент

Клиент для взаимодействия с серверным API реализует автоматическое обновление токена: при получении ответа 401 выполняется попытка обновления `access token` через `/auth/refresh`; если `refresh-токен` также недействителен, пользователь разлогинивается и перенаправляется на страницу входа.

Для повышения надёжности реализован механизм повторных попыток: до 3 попыток с экспоненциальной задержкой (500 мс базовая задержка + случайный джиттер от 0 до 500 мс) для ответов с кодом 5xx. Джиттер предотвращает синхронизацию повторных попыток от нескольких клиентов.

API-клиент разделён на 23 модуля по доменным областям, каждый из которых предоставляет типизированные методы для выполнения CRUD-операций.

## 2.4.4 Компоненты пользовательского интерфейса

Библиотека переиспользуемых компонентов включает `DataTable` — таблицу данных с сортировкой по столбцам, серверной пагинацией и адаптивным отображением на мобильных устройствах (на маленьких экранах таблица преобразуется в карточки); `Modal` и `ConfirmDialog` — модальные окна для создания, редактирования и подтверждения действий с поддержкой клавиши `Escape`; `FormField` — компонент формы с поддержкой валидации, отображения ошибок и меток; `FilterBar` — панель фильтров для таблиц с поддержкой текстового поиска, выпадающих списков и диапазонов дат;

Pagination — компонент пагинации с отображением номера текущей страницы и общего количества; TabBar — панель вкладок для переключения между разделами одной страницы; Toaster — уведомления (success/error/warning/info) с раскрываемыми подробностями и автоматическим скрыванием; MilkChart, LactationCurveChart, ReportChart, SeasonalChart — компоненты визуализации данных на основе Chart.js с адаптивными размерами; OfflineIndicator — индикатор состояния сети, отображающийся при потере соединения.

Валидация форм реализована на основе Svelte 5 Runes — реактивной системы состояния, позволяющей декларативно описывать правила валидации и отслеживать состояние формы (dirty, errors, touched). Правила валидации включают проверку обязательных полей, минимальной и максимальной длины, формата даты и числовых значений.

#### **2.4.5 Генерация отчётов**

Система поддерживает генерацию отчётов более 17 типов, покрывающих все аспекты управления стадом. Каждый отчёт доступен в формате JSON для отображения в интерфейсе, а также поддерживает экспорт в CSV и PDF.

Ключевые типы отчётов перечислены в таблице 2.



Таблица 2 – Основные типы отчётов системы

| Код    | Название          | Описание                                       |
|--------|-------------------|------------------------------------------------|
| R12    | Здоровье вымени   | Рабочий список коров с отклонениями за 24 часа |
| R13    | Неудачные доения  | Визиты с неуспешным доением                    |
| R16    | Обзор стада       | Дневная производительность стада               |
| R18    | Остаток корма     | Невыданный корм по коровам                     |
| R20    | Надой по времени  | Суточный надой с разбивкой по показателям      |
| R23    | Анализ вымени     | Расширенный анализ за 14 дней                  |
| R24    | Активность/жвачка | Здоровье, активность и жвачка                  |
| R31-34 | Календарь         | Ожидаемые отёлы, запуски, охота, стельность    |
| R41    | Эффективность     | Эффективность корова-робот                     |
| R52    | Лактация          | Кривые лактации по номерам                     |
| R56    | Робот             | Производительность роботов                     |
| R70    | Корм по типам     | Стоимость и объём кормов по типам              |
| R72    | Корм на корову    | Потребление корма на корову                    |

## 2.5 Модуль машинного обучения (Analytics-ML)

Модуль машинного обучения реализован на Python с использованием FastAPI и XGBoost. Он предоставляет API для прогнозирования и обнаружения аномалий, используемый серверной частью. На рисунке 5

представлена архитектура модуля машинного обучения.

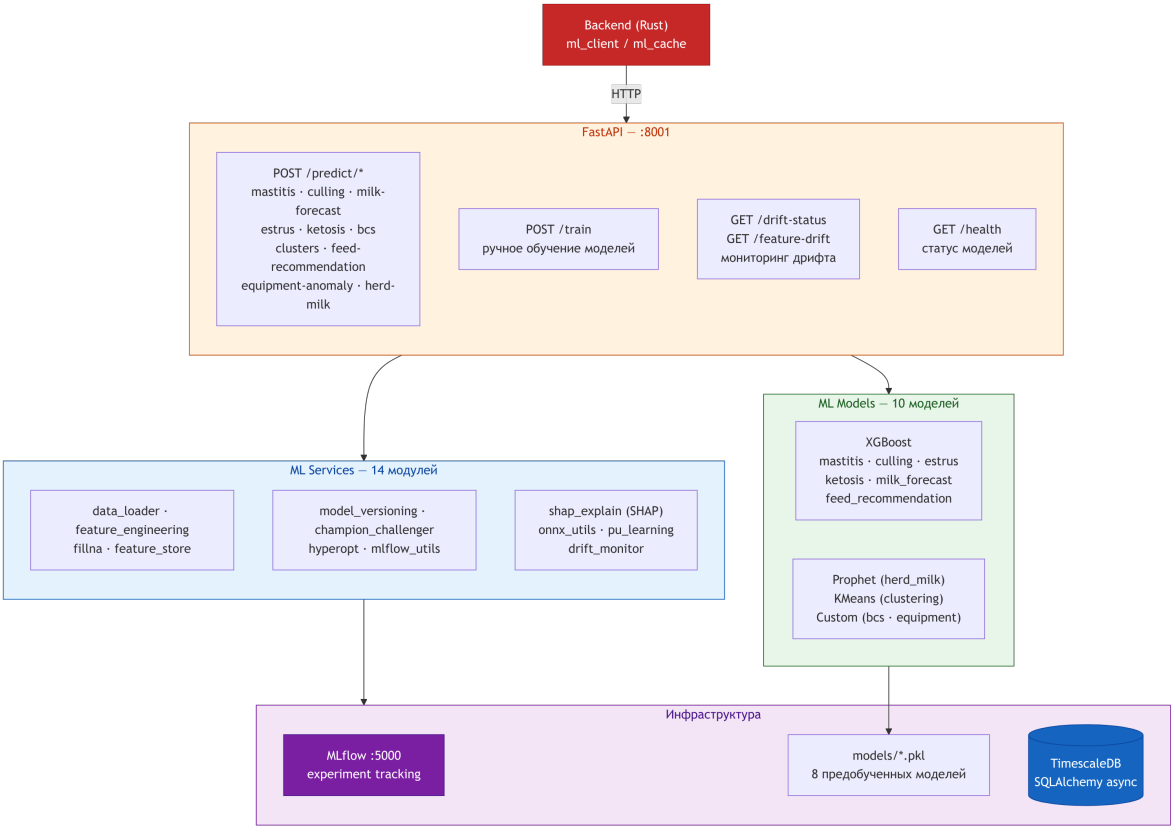


Рисунок 5 – Архитектура модуля машинного обучения

2.5.1 Модели машинного обучения

Разработано 11 моделей машинного обучения. В таблице 3 приведено их краткое описание.

Таблица 3 – Модели машинного обучения

| Модель                  | Алгоритм         | Тип           | Назначение                                         |
|-------------------------|------------------|---------------|----------------------------------------------------|
| Прогноз надоев (корова) | XGBoost          | Регрессия     | Дневной надой с q10/q90 доверительными интервалами |
| Прогноз надоев (стадо)  | Prophet          | Регрессия     | Тренд надоя стада с учётом сезонности              |
| Риск мастита            | XGBoost          | Классификация | Вероятность мастита (высокая/средняя/низкая)       |
| Предупреждение кетоза   | XGBoost          | Классификация | Риск метаболического нарушения                     |
| Обнаружение охоты       | XGBoost          | Классификация | Состояние (в охоте/приближается/нет)               |
| Риск выбраковки         | XGBoost          | Регрессия     | Дни до выбраковки                                  |
| Оценка BCS              | XGBoost          | Регрессия     | Оценка упитанности (1.0–5.0)                       |
| Кластеризация стада     | KMeans           | Кластеризация | Сегментация стада                                  |
| Аномалии оборудования   | Isolation Forest | Аномалии      | Аномалии работы роботов                            |
| Рекомендации по корму   | XGBoost          | Регрессия     | Рекомендуемый объём корма                          |

**Прогноз надоев на корову.** Модель XGBoost с квантильной регрессией предсказывает дневной надой с доверительными интервалами. Целевая переменная — объём молока (кг) за сутки. Признаки включают: надои за предыдущие 7, 14, 30 дней; скользящие среднее, стандартное отклонение, минимум, максимум за окна 7 и 14 дней; DIM (дни лактации); номер лактации; FPR; сезонные признаки (месяц, день недели).

Для построения доверительного интервала обучаются три модели: для медианы (q50), нижней границы (q10) и верхней границы (q90). Объективная функция Quantile Loss (пинбольная функция потерь) для квантиля  $\alpha$  определяется как:

$$L_{\alpha}(y, \hat{y}) = \begin{cases} \alpha \cdot (y - \hat{y}), & \text{если } y \geq \hat{y} \\ (1 - \alpha) \cdot (\hat{y} - y), & \text{если } y < \hat{y} \end{cases}$$

где  $y$  — фактическое значение,  $\hat{y}$  — прогноз.

**Прогноз надоев на стадо.** Модель Prophet [12] используется для прогнозирования надоев на уровне стада. Модель декомпозирует временной ряд на три компоненты:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$

где  $g(t)$  — тренд (кусочно-линейная или логистическая функция),  $s(t)$  — сезонная компонента (ряд Фурье),  $h(t)$  — эффект праздников и особых дней,  $\varepsilon_t$  — остатки. Prophet автоматически обнаруживает точки изменения тренда и устойчив к пропущенным данным и выбросам.

**Риск мастита.** Модель XGBoost Classifier классифицирует вероятность мастита [20] (высокая/средняя/низкая) на основе SCC, электрической проводимости молока, FPR и изменений в надоях. Ключевым признаком является динамика SCC: резкое увеличение (более чем на 50 процентов за сутки) является сильным индикатором воспаления вымени.

**Предупреждение кетоза.** Модель определяет риск метаболического нарушения через FPR, DIM, надой и потребление корма. Кетоз характерен для первых 60 дней лактации, когда потребность в энергии превышает потребление. Жирово-белковое соотношение (FPR) выше 1.4 является индикатором субклинического кетоза.

**Обнаружение охоты.** Модель классифицирует состояние коровы [1] на основе активности (шаги), жвачки (минуты), надоев и визитов. Коровы в охоте демонстрируют повышенную активность (в 3–5 раз выше базовой), снижение жвачки и увеличение количества визитов на доение.

**Риск выбраковки.** Модель XGBoost Regressor прогнозирует количество дней до выбраковки на основе возраста, надоев, SCC, номера лактации и индекса здоровья. Целевая переменная вычисляется как разница между текущей датой и датой фактической выбраковки для животных с известным исходом.

**Оценка упитанности (BCS).** Модель оценивает BCS (1.0–5.0) на основе веса, надоев, DIM, жвачки и эффективности кормления (надой/потребление корма). BCS — важный показатель здоровья, однако его ручная оценка трудоёмка и субъективна, что делает автоматизированную оценку ценной для фермера.

**Кластеризация стада.** Алгоритм KMeans [34] сегментирует стадо на группы с похожими характеристиками. Число кластеров  $k$  выбирается автоматически по метрике силуэта:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

где  $a(i)$  — среднее расстояние от объекта  $i$  до объектов того же кластера,  $b(i)$  — среднее расстояние от объекта  $i$  до объектов ближайшего другого кластера. Значение силуэта находится в диапазоне  $[-1, 1]$ , при этом значение ближе к 1 указывает на правильную кластеризацию.

**Аномалии оборудования.** Алгоритм Isolation Forest [35] обнаруживает аномалии в работе доильных роботов (скорость доения, время по четвертям, частота отказов). Алгоритм основан на принципе, что аномальные точки легче изолировать: случайное разбиение пространства признаков требует fewer разбиений для изоляции аномалий.

**Рекомендации по кормлению.** Модель XGBoost рекомендует объём корма на основе надоев, DIM, номера лактации и текущего потребления. Базовая формула для расчёта сухого вещества (СВ):

$$\text{СВ} = \text{Базовая\_потребность} + 0.45 \cdot \text{Надой\_кг}$$

Модель корректирует базовую формулу с учётом номера лактации (первотелки требуют больше энергии), DIM (пик лактации требует максимального потребления) и текущего BCS.

### 2.5.2 Инженерия признаков

Для каждой модели формируется набор признаков (features) на основе данных из PostgreSQL. Признаки включают лаговые (надои за предыдущие 7, 14, 30 дней), скользящие статистики (среднее, стандартное отклонение, минимум, максимум за окна 7, 14, 30 дней), производные метрики (FPR, абсолютное и процентное изменение надоев, DIM, номер лактации) и сезонные признаки (месяц, день недели).

Обработка пропущенных значений выполняется стратегией заполнения, зависящей от типа признака: медиана для числовых, мода для категориальных, ноль для счётчиков. Для лаговых признаков у новых

животных (с недостаточной историей) пропущенные значения заполняются медианой по стаду.

### 2.5.3 MLOps: обучение, переобучение и мониторинг

Автоматизация жизненного цикла моделей охватывает несколько аспектов. Автообучение запускается при отсутствии файла модели при запуске сервиса; минимальное количество записей для обучения — 100. Плановое переобучение выполняется через APScheduler с настраиваемым расписанием (по умолчанию еженедельно, в воскресенье в 02:00). Оптимизация гиперпараметров осуществляется библиотекой Optuna [25] с TPE-сэмплером (Tree-structured Parzen Estimator) в 8-мерном пространстве (learning\_rate, max\_depth, n\_estimators, min\_child\_weight, subsample, colsample\_bytree, reg\_alpha, reg\_lambda). Мониторинг дрейфа отслеживает prediction drift (z-среднее прогнозов > 2.0) и feature drift (z-среднее признаков > 2.5); при обнаружении дрейфа модель помечается для скорого переобучения. Версионирование обеспечивает хранение версий моделей в файловой системе с возможностью отката. Объяснимость прогнозов реализована через SHAP [36] (SHapley Additive exPlanations), показывающий вклад каждого признака. Отслеживание экспериментов осуществляется через интеграцию с MLflow [37] для логирования параметров обучения, метрик качества и артефактов моделей.

## 2.6 Предиктивная аналитика в серверной части

Помимо модуля машинного обучения, серверная часть содержит собственные алгоритмы аналитики, реализованные на Rust.

**Метод Хольта–Винтерса** [38] — тройное экспоненциальное сглаживание для прогнозирования временных рядов надоев. Модель учитывает три компоненты: уровень (level), тренд (trend) и сезонность (seasonality). Рекуррентные формулы:

$$l_t = \alpha \cdot (y_t - s_{t-m}) + (1 - \alpha) \cdot (l_{t-1} + b_{t-1}) \quad (1)$$

$$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1} \quad (2)$$

$$s_t = \gamma \cdot (y_t - l_t) + (1 - \gamma) \cdot s_{t-m} \quad (3)$$

где  $l_t$  — уровень,  $b_t$  — тренд,  $s_t$  — сезонная компонента,  $m$  — период сезонности (7 дней для надоя),  $\alpha, \beta, \gamma$  — параметры сглаживания. Прогноз на  $h$  шагов вперёд:

$$\hat{y}_{t+h} = l_t + h \cdot b_t + s_{t+h-m}$$

Для обнаружения выбросов используется межквартильный размах (IQR): значение считается выбросом, если оно выходит за пределы  $[Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}]$ . Для обнаружения структурных сдвигов применяется t-критерий: если среднее последних 7 дней значительно отличается от среднего предыдущих 14 дней ( $p\text{-value} < 0.05$ ), фиксируется структурный сдвиг.

**Кривая лактации Вуда** [39] — лог-линейная модель, описывающая зависимость надоя от дня лактации:

$$y(t) = a \cdot t^b \cdot e^{-ct}$$

Логарифмирование приводит к линейному виду:

$$\ln y(t) = \ln a + b \cdot \ln t - c \cdot t$$

Параметры  $\ln a$ ,  $b$  и  $c$  оцениваются методом наименьших квадратов. Модель позволяет прогнозировать надой на основе дня лактации, определять день пика лактации  $t_{\max} = -b/c$  и сравнивать фактические надой с ожидаемыми для выявления отклонений.

**Индекс здоровья** — составной показатель на основе z-оценок:

$$\text{Health Index} = -\frac{1}{n} \sum_{i=1}^n |z_i|$$

где  $z_i = (x_i - \mu_i)/\sigma_i$  — стандартизованное отклонение  $i$ -го показателя (надой, SCC, активность, жвачка) от среднего по стаду. Индекс принимает значения от  $-\infty$  до 0, при этом значения ниже  $-2$  указывают на потенциальные проблемы со здоровьем.

**Оптимизатор запуска** — определяет оптимальную дату сухостойного периода на основе прогноза лактации и планового отёла. Стандартный период запуска — 60 дней до отёла. Оптимизатор корректирует дату с учётом текущего надоя: если надой выше порога (например, 10 кг/день), запуск может быть сдвинут на более позднюю дату.

### 2.6.1 Сравнение моделей прогнозирования временных рядов

Для повышения точности прогнозирования надоев в серверной части реализована система автоматического сравнения 8 моделей прогнозирования временных рядов. Система позволяет для каждой коровы выбрать модель, лучше всего соответствующую её индивидуальной динамике надоев.

Исторические данные (90 дней) разбиваются на обучающую (80 процентов) и тестовую (20 процентов) выборки хронологически. Каждая модель обучается на обучающей выборке, после чего вычисляются метрики качества прогноза на тестовой выборке:

**MAPE** (Mean Absolute Percentage Error) — средняя абсолютная процентная ошибка:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$$

Значение MAPE менее 10 процентов означает отличное качество прогноза, 10–20 процентов — хорошее, более 20 процентов — неудовлетворительное. MAPE выбран как основная метрика, поскольку она инвариантна к масштабу (надой разных коров могут отличаться в 2–3 раза).

**RMSE** (Root Mean Squared Error) — среднеквадратичная ошибка (в литрах):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE дополняет MAPE, показывая типичное отклонение прогноза в абсолютных единицах. Больше, чем MAPE, штрафование крупных ошибок



делает RMSE чувствительным к выбросам.

Лучшей моделью признаётся та, которая демонстрирует минимальный MAPE на тестовой выборке. Данный подход основан на предположении, что модель, лучше работающая на недавних данных, будет лучше прогнозировать и будущее.

Реализованы следующие модели:

**1. SES (Simple Exponential Smoothing)** — простое экспоненциальное сглаживание. Модель поддерживает только уровень (level) и не учитывает тренд или сезонность. Рекуррентная формула:

$$\hat{y}_{t+1} = \alpha \cdot y_t + (1 - \alpha) \cdot \hat{y}_t$$

где  $\alpha \in (0, 1)$  — параметр сглаживания, оптимизируемый перебором по сетке  $\{0.05, 0.10, \dots, 0.95\}$  с выбором минимального SSE (sum of squared errors). Для устранения плоского прогноза вычисляется подразумеваемый наклон (implied slope) по последним fitted-значениям и экстраполируется линейно.

**2. SMA (Simple Moving Average)** — простое скользящее среднее с окном  $w$  (по умолчанию 7 дней). Прогноз — среднее последних  $w$  наблюдений. Как и SES, дополняется implied slope для формирования наклонного прогноза.

**3. WMA (Weighted Moving Average)** — взвешенное скользящее среднее. Веса назначаются линейно возрастающими:  $w_i = i$  для  $i = 1, \dots, w$ , что придаёт большую значимость недавним наблюдениям. Прогноз:

$$\hat{y}_{t+1} = \frac{\sum_{i=1}^w w_i \cdot y_{t-w+i}}{\sum_{i=1}^w w_i}$$

**4. Linear Regression** — линейная регрессия по времени. Модель оценивает параметры  $\hat{y}_t = a + b \cdot t$  методом наименьших квадратов. Подходит для данных с выраженным линейным трендом, но не улавливает сезонность.

**5. Holt's Linear Method** — двойное экспоненциальное сглаживание, добавляющее компоненту тренда к SES:

$$l_t = \alpha \cdot y_t + (1 - \alpha) \cdot (l_{t-1} + b_{t-1}) \quad (4)$$

$$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1} \quad (5)$$

Прогноз:  $\hat{y}_{t+h} = l_t + h \cdot b_t$ . Параметры  $\alpha, \beta$  оптимизируются jointly по сетке  $10 \times 10$  значений.

**6. Holt-Winters** — тройное экспоненциальное сглаживание

(см. формулы (1)–(3) выше). Добавляет сезонную компоненту с периодом  $m = 7$  дней (недельная сезонность надоев). Наиболее гибкая модель, способная улавливать уровень, тренд и сезонность одновременно.

**7. Fourier Series** — модель на основе рядов Фурье с линейным трендом. Данные сначала детрендируются (линейная регрессия), затем остатки аппроксимируются суммой синусов и косинусов с фиксированным базовым периодом 7 дней (недельная сезонность):

$$\hat{y}_t = (a + b \cdot t) + \sum_{k=1}^K \left[ A_k \cos\left(\frac{2\pi t}{T_k}\right) + B_k \sin\left(\frac{2\pi t}{T_k}\right) \right]$$

где  $T_k = 7/k$  — период  $k$ -й гармоники,  $K$  — количество гармоник (по умолчанию 3). Коэффициенты  $A_k$ ,  $B_k$  оцениваются методом наименьших квадратов. Модель хорошо улавливает периодические паттерны, но может экстраполировать тренд слишком агрессивно.

**8. ARIMA(0,1,1)** — модель авторегрессии интегрированного скользящего среднего. Данная спецификация эквивалентна экспоненциальному сглаживанию с дрейфом. Модель работает с первыми разностями  $\Delta y_t = y_t - y_{t-1}$ :

$$\Delta \hat{y}_t = \mu + \theta \cdot \varepsilon_{t-1}$$

где  $\mu$  — средний дрейфт (среднее приращение),  $\theta$  — параметр МА(1),  $\varepsilon_{t-1}$  — предыдущая ошибка. Параметр  $\theta$  оптимизируется перебором по сетке  $\{0.00, 0.05, \dots, 0.95\}$ . Для прогноза на  $h \geq 2$  шагов вклад МА-члена равен нулю, и прогноз вырождается в прямую с наклоном  $\mu$ .

Интерфейс системы отображает сравнительную таблицу моделей с их MAPE и RMSE, а также интерактивный график, позволяющий пользователю переключаться между моделями и визуально оценивать качество подгонки (fitted values) и прогноза.

### 2.6.2 Ансамблевый прогноз

Для повышения робастности прогнозирования реализован ансамблевый эндпоинт `/analytics/ensemble-forecast`, объединяющий прогноз ML-сервиса (XGBoost) и лучшую статистическую модель (Rust) методом взвешенного среднего. Вес каждой модели определяется обратно пропорционально её MAPE:

$$w_{\text{ML}} = \frac{1/\text{MAPE}_{\text{ML}}}{1/\text{MAPE}_{\text{ML}} + 1/\text{MAPE}_{\text{Rust}}}, \quad w_{\text{Rust}} = 1 - w_{\text{ML}}$$

Итоговый прогноз для дня  $t$ :

$$\hat{y}_t = w_{\text{ML}} \cdot \hat{y}_t^{\text{ML}} + w_{\text{Rust}} \cdot \hat{y}_t^{\text{Rust}}$$

ML-сервис имеет оценочный MAPE 8 процентов (настройка по умолчанию), тогда как MAPE Rust-модели вычисляется динамически для каждого животного. Если ML-сервис недоступен, система автоматически откатывается к прогнозу лучшей Rust-модели с весом 1.0. Интерфейс отображает веса обеих моделей, а также вклад каждой в итоговый прогноз, что повышает прозрачность и доверие фермера к системе.

### 2.6.3 Объяснимость прогнозов (SHAP)

Для обеспечения интерпретируемости прогнозов ML-сервис использует метод SHAP [36] (SHapley Additive exPlanations), основанный на теории кооперативных игр Шепли. Для каждого прогноза надоя вычисляются SHAP-значения, показывающие вклад каждого признака в отклонение от базового значения модели:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

где  $\phi_i$  — SHAP-значение признака  $i$ ,  $F$  — множество всех признаков,  $f(S)$  — прогноз модели на подмножестве признаков  $S$ . Для XGBoost используется оптимизированный TreeExplainer, вычисляющий точные SHAP-значения за линейное время.

SHAP-объяснения включены в ответы пяти моделей прогнозирования (мастит, кетоз, охота, выбраковка, BCS) и модели прогноза надоев. В интерфейсе на странице животного SHAP-значения визуализируются в виде горизонтальной столбчатой диаграммы с цветовой кодировкой: зелёный для положительного вклада (увеличивает прогноз), красный для отрицательного (уменьшает). Это позволяет фермеру понять, какие именно факторы (например, высокий SCC, снижение жвачки) повлияли на конкретный прогноз.

## **2.7 Мониторинг**

Стек мониторинга реализован на базе Grafana LGTM (Loki, Grafana, Tempo, Mimir/Prometheus). На рисунке 6 представлена архитектура стека мониторинга.

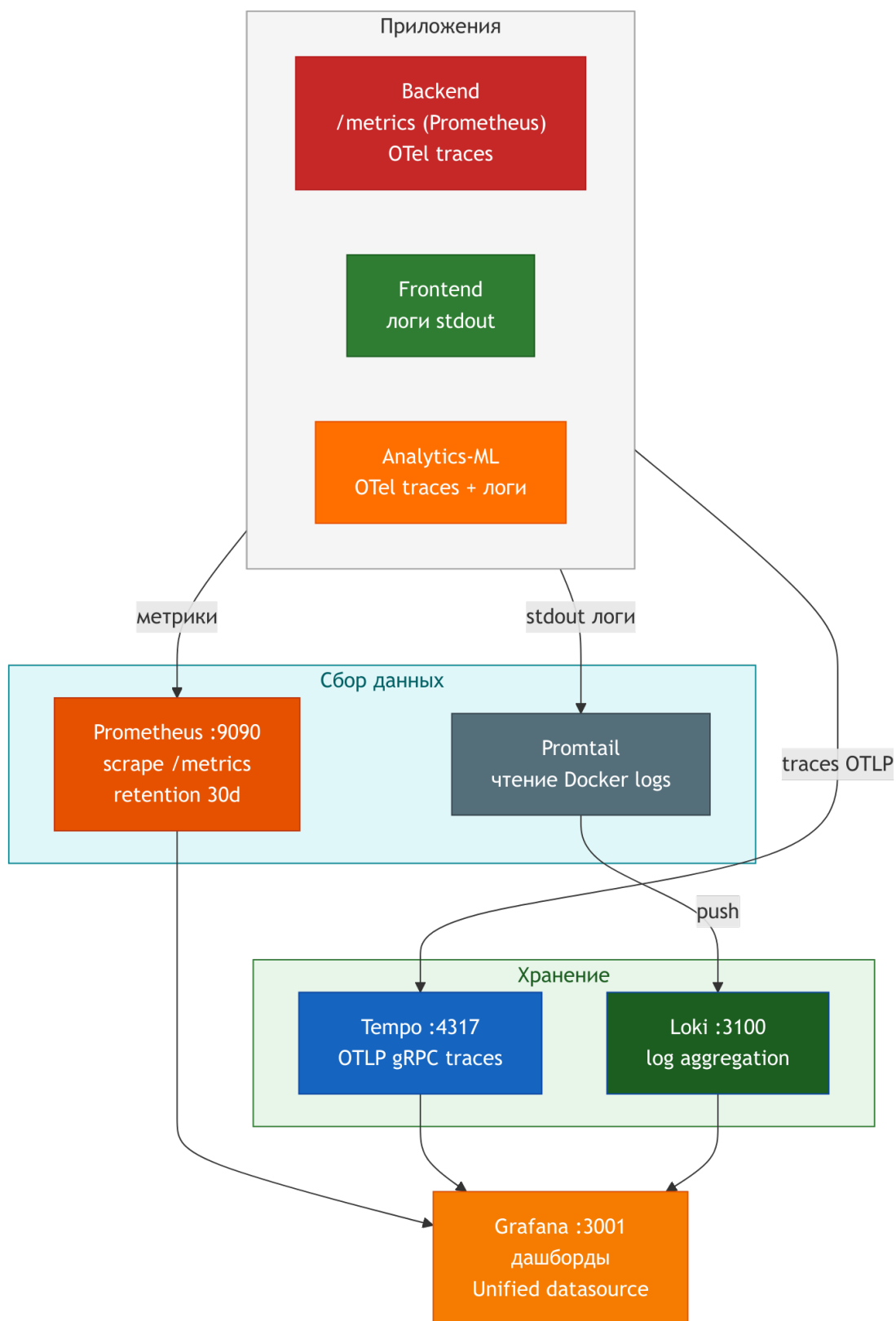


Рисунок 6 – Архитектура стека мониторинга

Prometheus [17] обеспечивает сбор и хранение метрик с 30-дневным хранением: счётчики HTTP-запросов (`http_requests_total`), гистограммы длительности запросов

(`http_request_duration_seconds`), метрики подключения к БД и Redis. Grafana [18] визуализирует дашборды: преднастроенный дашборд backend отображает частоту запросов, время ответа, процент ошибок и использование памяти. Loki [26] агрегирует логи: Promtail собирает логи из Docker-контейнеров, парсит JSON-формат (уровень, сообщение, цель) и отправляет в Loki с хранением 168 часов и компактизацией. Tempo [27] обеспечивает распределённую трассировку через OpenTelemetry OTLP/gRPC, позволяя отследить путь запроса от Nginx через Backend до PostgreSQL.

## 2.8 Непрерывная интеграция и доставка (CI/CD)

Конвейер CI/CD реализован на GitHub Actions [19]. На рисунке 7 представлена схема конвейера непрерывной интеграции и доставки.

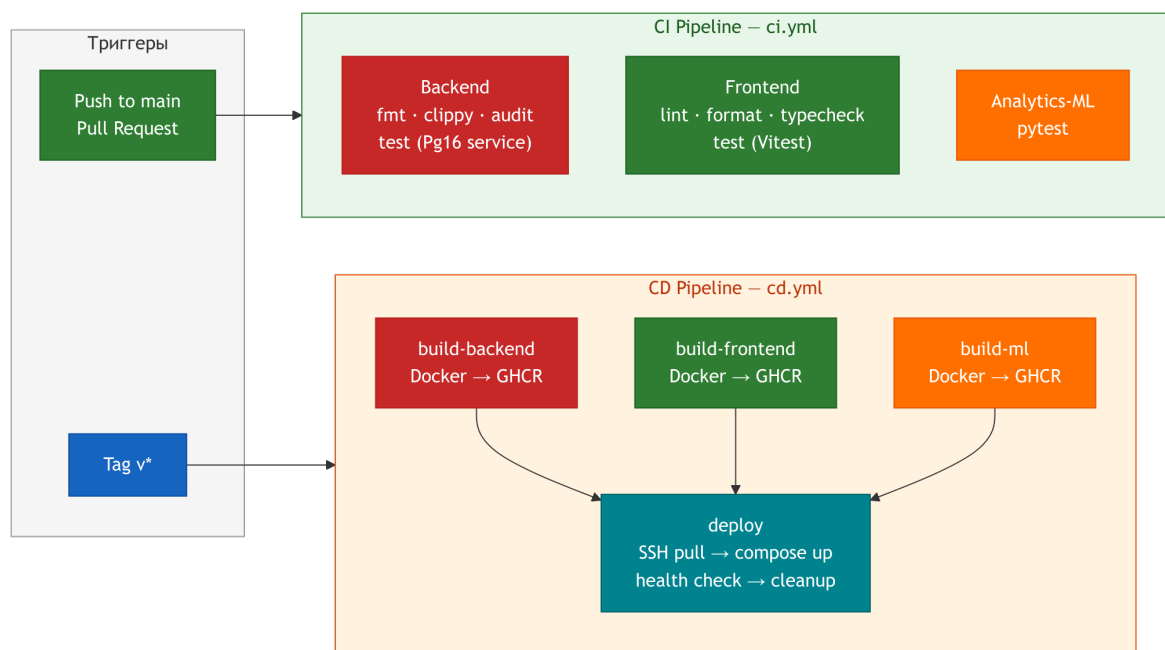


Рисунок 7 – Конвейер непрерывной интеграции и доставки

Конвейер включает:

**Непрерывная интеграция** запускается при каждом push и pull request. Три параллельные задачи: для backend на Rust выполняются проверка форматирования (`cargo fmt --check`), статический анализ (`cargo clippy -D warnings`), аудит зависимостей (`cargo audit`) и запуск тестов (`cargo test`) с PostgreSQL в качестве сервисного контейнера; для frontend на Node.js 20 — установка зависимостей (`npm ci`), аудит уязвимостей (`npm audit --production --audit-level=high`),

линтинг (`npm run lint`), проверка форматирования (`npm run format:check`), проверка типов Svelte (`npm run check`) и запуск тестов Vitest; для `analytics-ml` на Python 3.12 — установка зависимостей и запуск тестов `pytest`.

**Непрерывная доставка** запускается при `push` в `main/master` или при создании тега `v*`. Три параллельные задачи сборки Docker-образов с использованием `Docker buildx` и кэша GitHub Actions. Образы публикуются в GHCR [40] с тегами `latest` и `<sha>`.

При создании тега версии дополнительно запускается задача деплоя: SSH на сервер, `pull` образов, создание тегов для отката, `docker compose -f docker-compose.prod.yml up -d`, проверка работоспособности (`health check`) в цикле: 5 попыток с интервалом 5 секунд.

## 2.9 Развёртывание

Для развёртывания используется Docker Compose с двумя конфигурациями. На рисунке 8 представлена схема развёртывания системы.

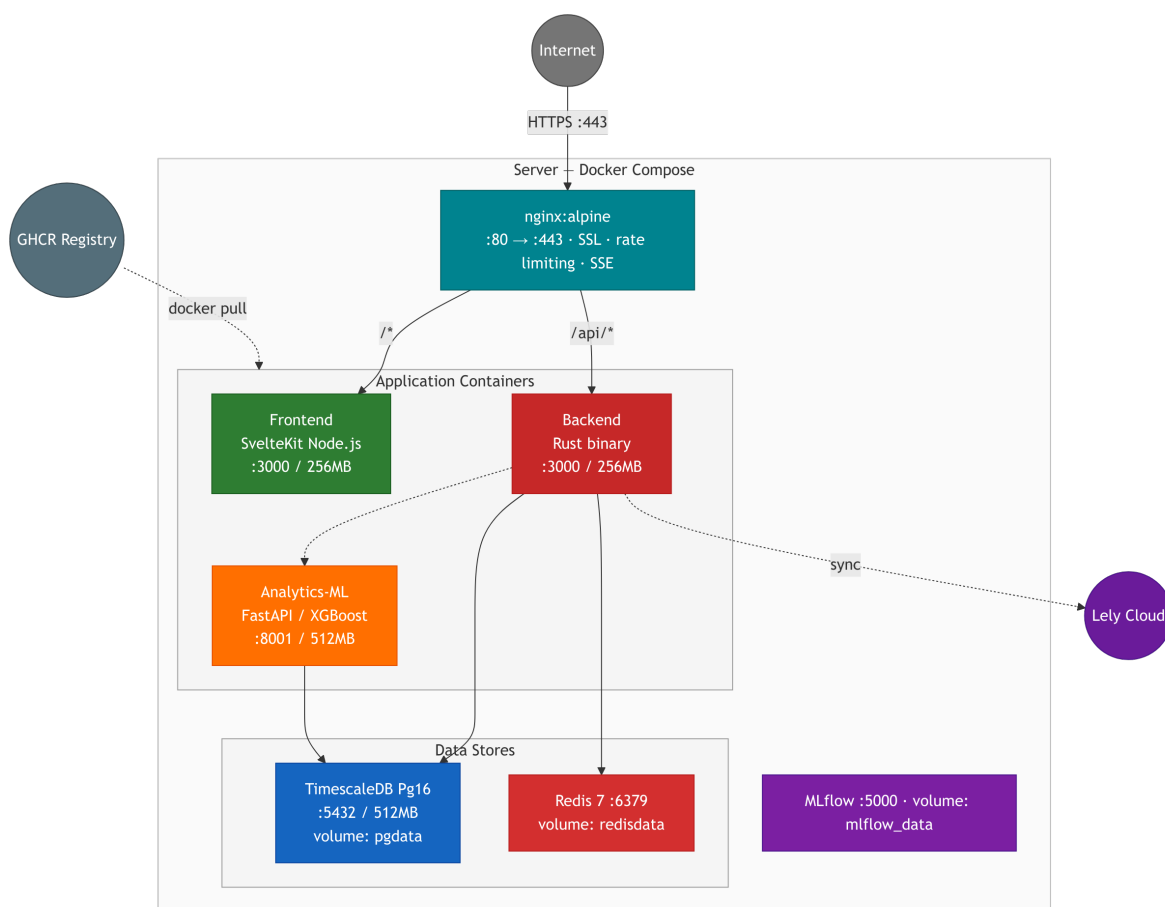


Рисунок 8 – Схема развёртывания системы

**Разработческая** (`docker-compose.yml`): 7 сервисов — PostgreSQL (TimescaleDB), Redis, backend, frontend, Nginx, analytics-ml, MLflow. Nginx на порту 8080. PostgreSQL инициализируется скриптом, создающим дополнительные базы данных для ML-сервиса.

**Продуктовая** (`docker-compose.prod.yml`): те же 7 сервисов с усилением безопасности: переменные окружения через env-файлы, обязательная валидация секретов (`POSTGRES_PASSWORD`, `JWT_SECRET`), `restart: unless-stopped`, ротация логов (10 МБ, 3 файла), ограничения памяти (256 МБ для backend/frontend, 512 МБ для PostgreSQL/analytics-ml), образы из GHCR, SSL-терминация через Nginx с монтированием сертификатов.

Nginx в продуктовой конфигурации обеспечивает: редирект HTTP → HTTPS, SSL (TLS 1.2/1.3), gzip-сжатие, заголовки безопасности (`X-Frame-Options: DENY`, HSTS, `X-XSS-Protection`), rate limiting (30 запросов/сек для API, 5 запросов/сек для auth), кэширование статических ресурсов (30 дней для immutable-ассетов).



## 3 ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РАБОТЫ СИСТЕМЫ

### 3.1 Стратегия тестирования

Тестирование разработанной информационной системы включает модульные тесты, интеграционные тесты, сквозные (E2E) тесты и нагрузочное тестирование. Тесты разделены по компонентам системы. Стратегия тестирования основана на пирамиде тестирования: большое количество быстрых модульных тестов, среднее количество интеграционных тестов и небольшое количество E2E-тестов.

В таблице 4 приведено общее количество тестов по компонентам.

Таблица 4 – Общее количество тестов по компонентам

| Компонент                | Инструмент               | Количество файлов |
|--------------------------|--------------------------|-------------------|
| Backend (Rust)           | sqlx::test + tower       | 17                |
| Frontend (TypeScript)    | Vitest + Testing Library | 10                |
| Analytics-ML (Python)    | pytest                   | 3                 |
| Нагрузочное тестирование | k6                       | 3                 |
| E2E                      | Playwright               | 1                 |

### 3.2 Тестирование серверной части

Интеграционные тесты серверной части реализованы с использованием макроса `sqlx::test`, автоматически подготавливающего тестовую базу данных с применением миграций. HTTP-запросы выполняются `in-process` через `tower::ServiceExt`, что исключает сетевые задержки и позволяет тестировать обработчики в изолированной среде.

Тестовый набор включает 17 файлов, покрывающих:

- аутентификацию и авторизацию — вход с валидными и невалидными учётными данными, выход, обновление токена, проверку ролей `admin/user`, ограничение частоты попыток входа (5 за 60 секунд), обязательную смену пароля;
- CRUD-операции для всех доменных областей — животные, надои, воспроизводство, кормление, контакты, местоположения, быки, переводы, запасы, включая пакетные операции и импорт CSV;
- генерацию отчётов более 17 типов и аналитику — KPI, тренды,

- прогнозы;
- г) интеграцию с Lely — конфигурация, статус синхронизации, ручной запуск;
  - д) граничные случаи — некорректные данные (невалидный JSON, отрицательные значения, строки превышающей длины); конфликты (дублирование инвентарных номеров); несуществующие ресурсы (404); несанкционированный доступ (401, 403).

Для каждого теста создаётся изолированная тестовая база данных, которая уничтожается после выполнения. Общий вспомогательный модуль (`common/mod.rs`) предоставляет фабрики для создания тестового состояния, генерации JWT-токенов и построения HTTP-запросов.

Пример структуры интеграционного теста: подготовка (создание тестового приложения с подключением к тестовой БД), выполнение (отправка HTTP-запроса к тестируемому эндпоинту), проверка (asserting статус-кода, структуры JSON-ответа и наличия ожидаемых данных) и очистка (тестовая БД уничтожается автоматически).

Помимо функциональных тестов, реализованы бенчмарки для оценки производительности: SQL-запросов (`bench_sql`), генерации PDF (`bench_pdf`), аутентификации (`bench_auth`) и rate limiting (`bench_rate_limit`). Бенчмарки используют критерий Criterion и запускаются командой `cargo bench`.

### 3.3 Тестирование клиентского приложения

Модульные тесты клиентского приложения реализованы с использованием Vitest [41] и `@testing-library/jest-dom`. Тестовый набор включает 10 файлов, покрывающих UI-компоненты (DataTable — сортировка, пагинация, рендеринг данных; Modal — открытие, закрытие, контент; FormField — валидация, отображение ошибок; Pagination — рендеринг номеров страниц) и утилиты (валидаторы обязательных полей, минимальной длины и формата email; форматирование дат и чисел; построение запросов с параметрами фильтрации и пагинации; хелперы графиков для конфигурации Chart.js; CRUD-модальные окна с состоянием создания и редактирования; бизнес-правила валидации).

Сквозное (E2E) тестирование реализовано с использованием Playwright. Тесты проверяют ключевые пользовательские сценарии: аутентификация

(вход с валидными и невалидными данными), навигация по основным разделам, CRUD-операции (создание, редактирование, удаление животного), экспорт отчётов в CSV и PDF.

### **3.4 Тестирование модуля машинного обучения**

Тесты модуля машинного обучения включают: `test_api.py` — тесты API-эндпоинтов (проверка `health`-эндпоинта, вызов прогнозов для всех 11 моделей с валидными и невалидными входными данными, запуск обучения); `test_models.py` — тесты моделей (обучение на синтетических данных, проверка формата прогноза на соответствие JSON-схеме, проверка доверительных интервалов  $q_{10} < q_{50} < q_{90}$ ); `test_drift_and_auth.py` — тесты мониторинга дрейфа (обнаружение аномального распределения прогнозов) и аутентификации по API-ключу (отклонение запросов без ключа или с невалидным ключом).

### **3.5 Нагрузочное тестирование**

Нагрузочное тестирование проведено с использованием `k6` [42]. Реализованы три сценария: дымовой тест (`smoke.js`) с 5 виртуальными пользователями и 10 итерациями для проверки `health`-эндпоинта и входа в систему; нагрузочный тест (`load.js`), моделирующий типичную рабочую нагрузку с постепенным увеличением числа пользователей от 10 до 50; стресс-тест (`stress.js`) для определения предельной нагрузки с увеличением числа пользователей до 200.

Метрики, собираемые в ходе нагрузочного тестирования: время ответа ( $p_{50}$ ,  $p_{90}$ ,  $p_{95}$ ), количество запросов в секунду (RPS), процент ошибок (HTTP 5xx), время ожидания соединения (connection time).

### **3.6 Демонстрация системы**

Для демонстрации работы системы используется скрипт `dev.sh`, автоматически запускающий PostgreSQL, разворачивающий тестовые данные и запускающий все компоненты системы. Тестовые данные включают 100 животных с историей надоев, воспроизводства и кормления за 365 дней.

### 3.6.1 Аутентификация

При первом запуске система создаёт пользователя `admin` со случайным паролем из 16 символов, который выводится в стандартный поток ошибок. При входе в систему, если установлен флаг обязательной смены пароля, пользователь перенаправляется на форму смены пароля. Минимальная длина пароля — 8 символов. Демонстрация интерфейса аутентификации представлена на рисунке 9.

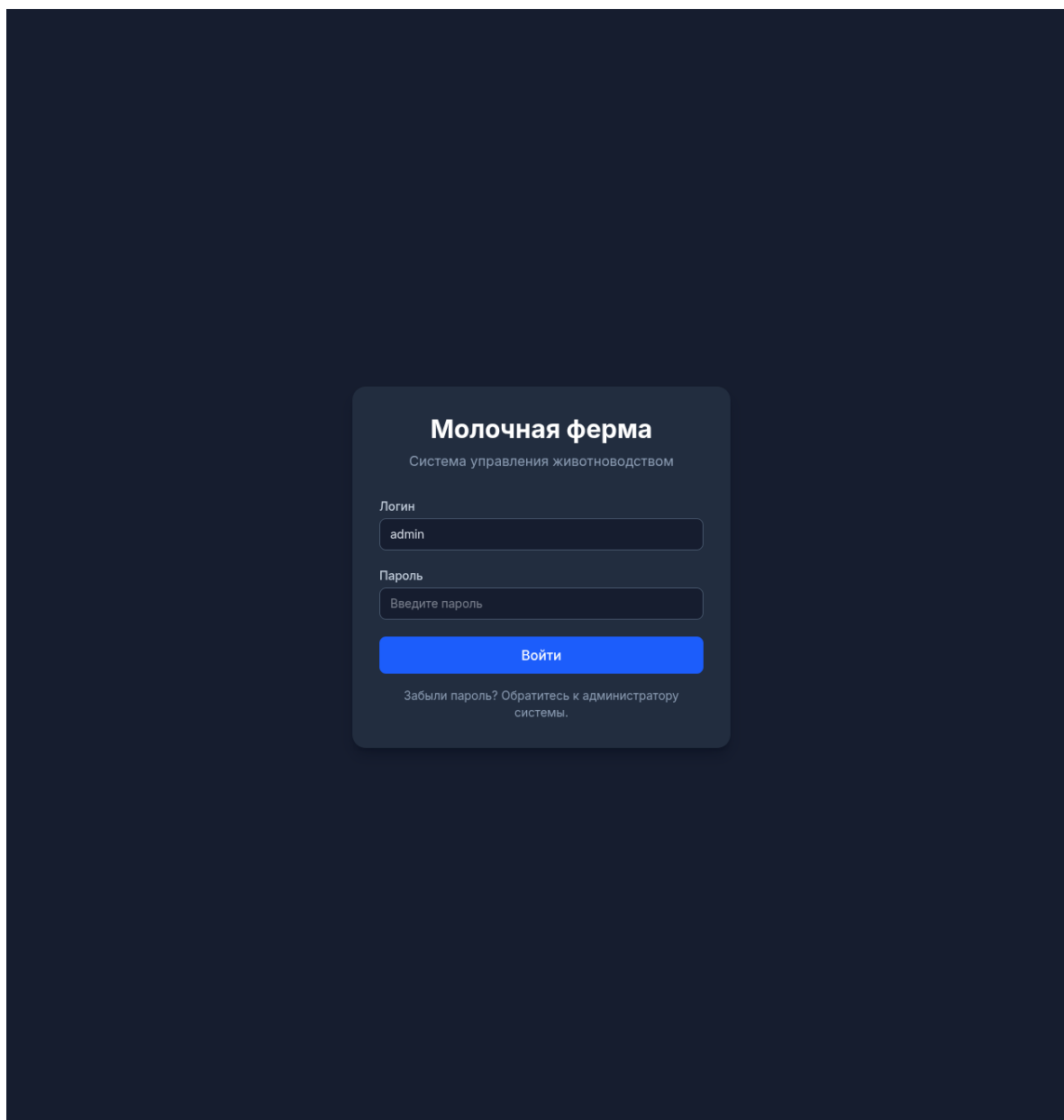


Рисунок 9 – Страница аутентификации

### 3.6.2 Дашборд

Главная страница системы — дашборд, отображающий ключевые показатели стада: количество животных (с разбивкой по полу и статусу), средний суточный надой, статус воспроизводства (количество стельных, в охоте, ожидаемых отёлов), количество активных оповещений. Виджеты настраиваются пользователем через панель настроек: можно выбрать отображаемые KPI и их расположение. Пример дашборда представлен на рисунке 10.

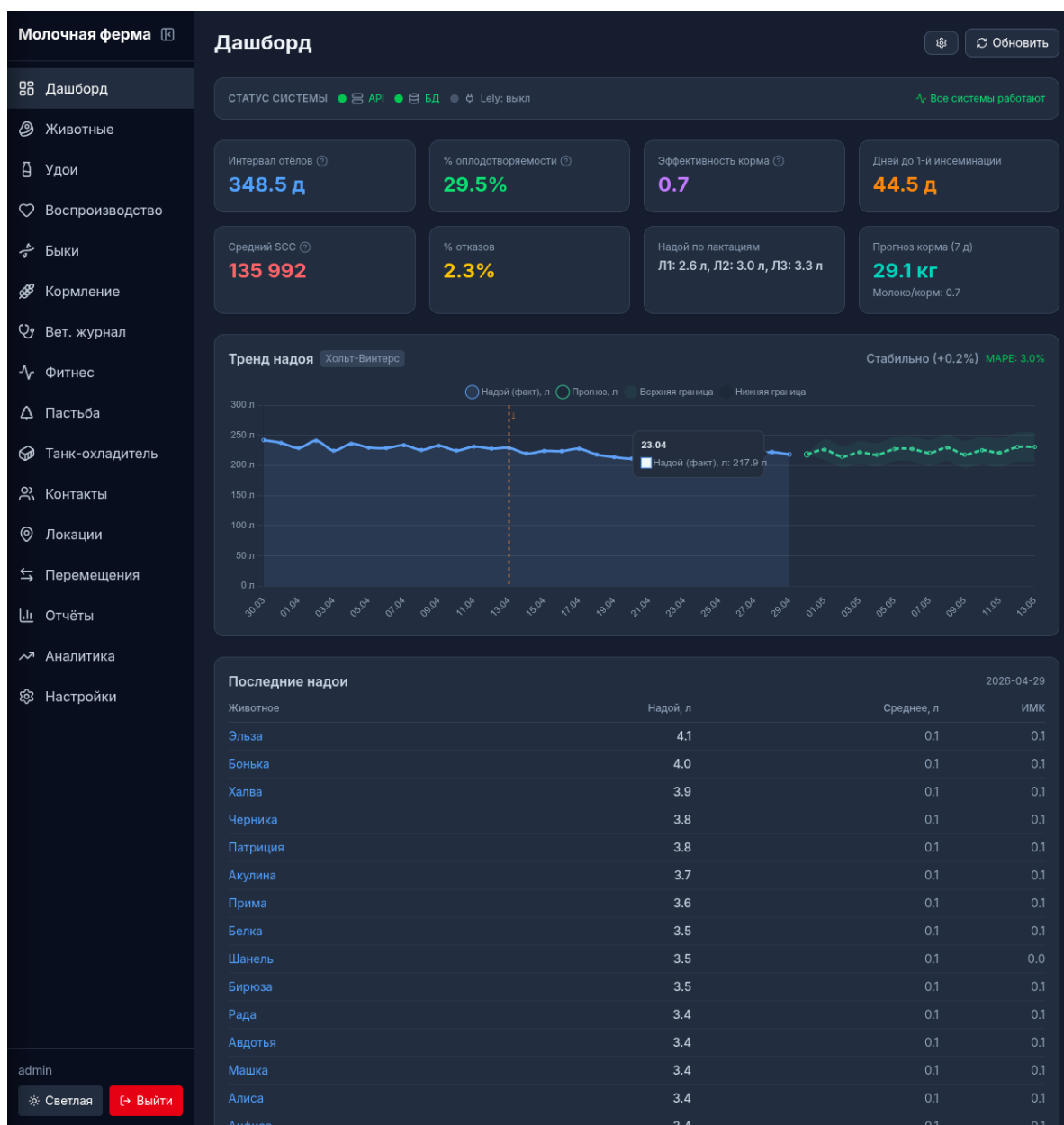


Рисунок 10 – Дашборд системы

### 3.6.3 Управление животными

Страница списка животных предоставляет таблицу с сортировкой по столбцам (кличка, инвентарный номер, пол, дата рождения, статус), фильтрацией (по полу, местоположению, дате рождения) и пагинацией (по 25, 50 или 100 записей). Поддерживается поиск по кличке и инвентарному номеру с использованием нечёткого поиска `pg_trgm` (триграммное сходство).

Для каждого животного доступна детальная карточка с полной информацией, разделённой на вкладки: биография (основные данные, происхождение), надои (графики и таблица), воспроизводство (история событий), кормление (план и факт), ветеринарные записи. Поддерживается экспорт карточки в PDF. Пример списка животных представлен на рисунке 11, пример детальной карточки — на рисунке 12.

Молочная ферма

Дашборд

Животные

Удои

Воспроизводство

Быки

Кормление

Вет. журнал

Фитнес

Пастъба

Танк-охладитель

Контакты

Локации

Перемещения

Отчёты

Аналитика

Настройки

admin

Светлая

Выйти

Животные

Импорт CSV

+

Добавить

Поиск

Имя, номер жизни, UCN...

Пол

Все

Статус

Активные

Найти

| №                                                                                                     | Имя      | Пол    | Дата рождения | Локация               | Группа | Статус  | Действия                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------|----------|--------|---------------|-----------------------|--------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  RU516729513849799   | Сюзи     | Корова | 2021-09-19    | Выгульная площадка Б  | 5      | Активно |       |
|  RU936951702510397   | Мони     | Корова | 2021-02-22    | Телятник              | 3      | Активно |       |
|  RU395447170702701   | Легенда  | Корова | 2022-11-05    | Телятник              | 4      | Активно |       |
|  RU253287255096351   | Бирюза   | Корова | 2020-10-30    | Выгульная площадка Б  | 1      | Активно |       |
|  RU593636038940345   | Дарёнка  | Корова | 2022-06-26    | Пастбище Южное        | 3      | Активно |       |
|  RU650819627598782   | Арина    | Корова | 2021-04-28    | Коровник 3 (новый)    | 1      | Активно |       |
|  RU841255232690143   | Афина    | Корова | 2021-10-07    | Пастбище Северное     | 6      | Активно |       |
|  RU898278843418946   | Снежинка | Корова | 2022-05-24    | Доильный зал          | 4      | Активно |       |
|  RU345755201375381   | Милашка  | Корова | 2020-10-27    | Карантин              | 3      | Активно |       |
|  RU466304314626388   | Леля     | Корова | 2020-12-08    | Пастбище Южное        | 4      | Активно |       |
|  RU136604541725455   | Пеструха | Корова | 2021-10-18    | Карантин              | 5      | Активно |       |
|  RU691012795309461   | Бритни   | Корова | 2020-10-31    | Родильное отделение   | 4      | Активно |       |
|  RU168712830236083   | Мадлен   | Корова | 2020-12-06    | Коровник 2            | 3      | Активно |       |
|  RU598595854359731  | Умка     | Корова | 2021-08-13    | Коровник 1 (основной) | 3      | Активно |     |
|  RU455000314815656 | Мальвина | Корова | 2021-06-11    | Коровник 1 (основной) | 6      | Активно |   |
|  RU89508514720773  | Арфа     | Корова | 2021-01-03    | Коровник 2            | 2      | Активно |   |
|  RU826898476995753 | Хлоя     | Корова | 2021-07-29    | Выгульная площадка Б  | 6      | Активно |   |
|  RU461100864372615 | Джесси   | Корова | 2021-12-25    | Коровник 3 (новый)    | 2      | Активно |   |
|  RU998092836813388 | Цыганка  | Корова | 2021-07-15    | Выгульная площадка Б  | 2      | Активно |   |
|  RU461931030472338 | Астра    | Корова | 2020-11-30    | Телятник              | 4      | Активно |   |

Рисунок 11 – Список животных

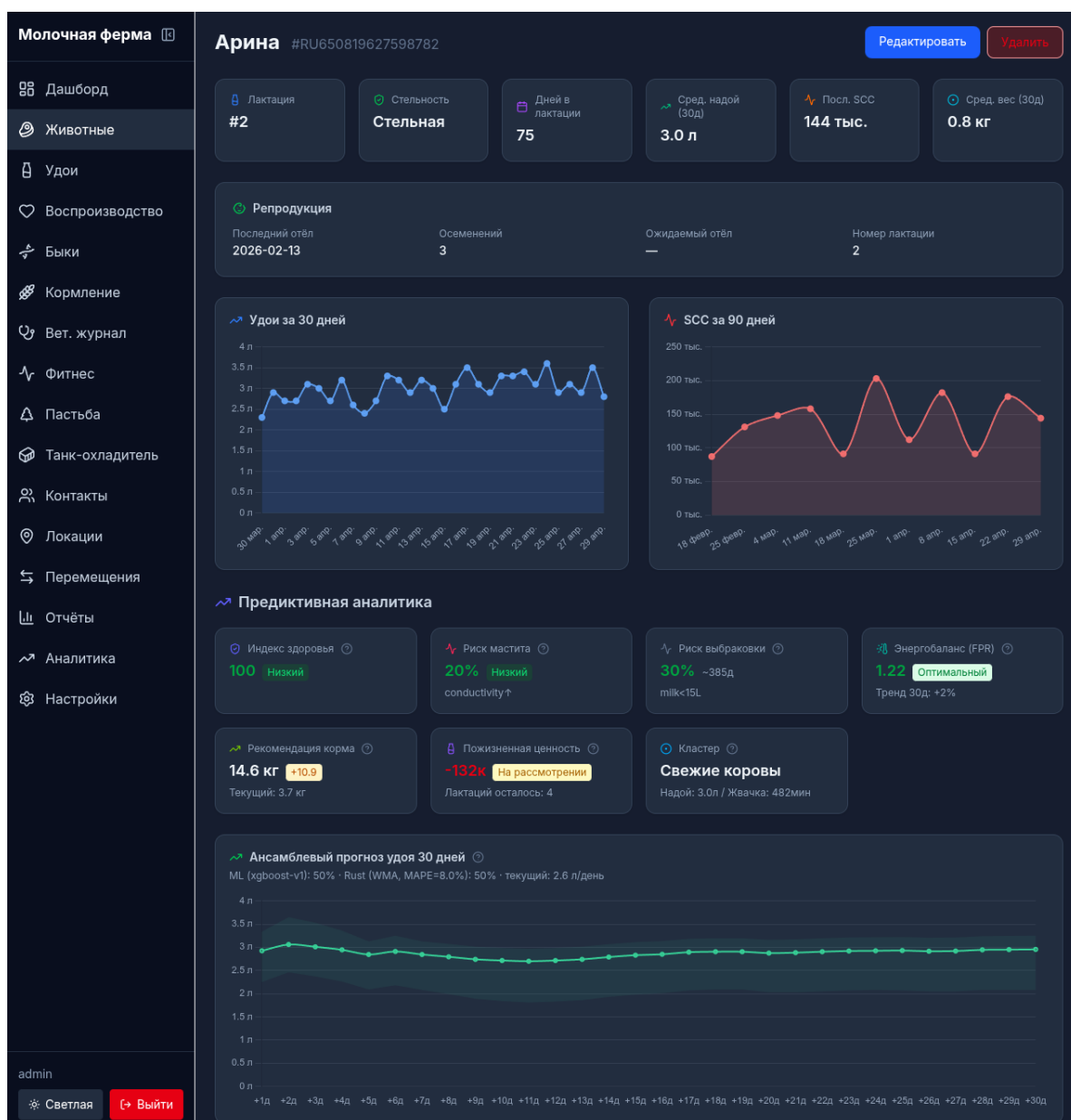


Рисунок 12 – Детальная карточка животного

### 3.6.4 Отчёты и аналитика

Система генерирует более 17 типов отчётов (см. таблицу 2), каждый из которых отображается в виде таблицы и/или графика в интерфейсе. Отчёты поддерживают фильтрацию по дате и группе животных, а также экспорт в CSV и PDF.

Аналитическая панель отображает: ключевые показатели эффективности (KPI) — средний надой, процент стельных, интервал между отёлами; тренды надоев с прогнозом на основе модели Хольта–Винтерса; прогноз стельных коров; кривые лактации по номерам; кластеризацию стада.



### 3.6.5 Интеграция с Lely

При включённой интеграции с Lely система автоматически синхронизирует данные каждые 5 минут. Страница статуса синхронизации отображает дату последней успешной синхронизации для каждого из 23 типов данных и количество синхронизированных записей. Через интерфейс доступен ручной запуск синхронизации и просмотр логов синхронизации.

### 3.6.6 Оповещения

Модуль оповещений автоматически создаёт оповещения на основе настраиваемых пороговых значений. Проверка выполняется каждые 15 минут фоновым заданием. Отслеживаются следующие категории событий:

- а) падение надоев более чем на 20 процентов за сутки;
- б) высокий SCC (более 200 000 клеток/мл);
- в) снижение активности более чем на 30 процентов за сутки;
- г) снижение жвачки более чем на 25 процентов за сутки;
- д) ожидаемый отёл (за 7 дней до планируемой даты).

Пороговые значения настраиваются через страницу настроек без перезапуска сервера. Оповещения отображаются в интерфейсе с категоризацией по степени важности (high, medium, low). Поддерживается подтверждение оповещения (acknowledge) для отслеживания реакции фермера. Оповещения также могут быть отправлены через канал уведомлений (браузерные уведомления или Telegram-бот).

## 3.7 Мониторинг работоспособности

Стек Grafana LGTM обеспечивает комплексное наблюдение за системой. Grafana дашборды отображают частоту запросов, время ответа (p50, p95, p99), использование ресурсов (CPU, память) и ошибки по маршрутам. Loki позволяет искать логи по уровню (info, warn, error) и целевому сервису с использованием языка LogQL. Tempo обеспечивает распределённую трассировку запросов через цепочку Nginx → Backend → PostgreSQL/ML, позволяя идентифицировать медленные запросы. Prometheus собирает метрики с backend (/metrics), analytics-ml (/metrics) и Redis с интервалом 15 секунд.

Метрики backend включают: `http_requests_total` (счётчик,

разбитый по маршруту, методу и статус-коду),  
http\_request\_duration\_seconds (гистограмма),  
db\_connections\_active, rate\_limit\_rejections\_total.

## ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы бакалавра была разработана информационная система управления молочной фермой с интеграцией роботизированного доильного оборудования Lely и предиктивной аналитикой на основе моделей машинного обучения. Все поставленные задачи были выполнены.

Проведён анализ предметной области и существующих решений (Lely Horizon, DeLaval DelPro, SCR Heatime, 1С: Управление фермой), показавший, что ни одно из них не обеспечивает одновременно полный цикл управления стадом, интеграцию с оборудованием сторонних производителей, встроенные модели машинного обучения и открытую архитектуру. На основании анализа сформулировано техническое задание и обоснован выбор стека технологий (Rust/Axum, SvelteKit, Python/FastAPI, PostgreSQL/TimescaleDB).

Спроектирована и реализована трёхзвенная архитектура. Серверная часть на Rust/Axum включает 25 модулей обработчиков, 33 модуля сервисов, 24 модуля моделей данных и 6 модулей middleware. Аутентификация на основе JWT с HttpOnly cookie обеспечивает защиту от CSRF и XSS. Middleware-стек включает rate limiting (Redis/in-memory), gzip-сжатие, CSRF-защиту, метрики Prometheus и распределённую трассировку. База данных PostgreSQL содержит более 35 таблиц с расширением TimescaleDB для эффективной работы с временными рядами.

Реализована интеграция с облачным API Lely Horizon, обеспечивающая автоматическую двустороннюю синхронизацию 23 типов данных (животные, надои, воспроизводство, кормление, фитнес, оборудование) с шифрованием учётных данных AES-256-GCM и инкрементальной синхронизацией по временным диапазонам.

Клиентское веб-приложение на SvelteKit 5 содержит более 24 страниц с серверным рендерингом и адаптивным интерфейсом на Tailwind CSS. Разработано более 16 переиспользуемых UI-компонентов, включая DataTable с серверной пагинацией и адаптивным отображением, компоненты визуализации на Chart.js (надои, SCC, лактационные кривые, прогнозы), систему уведомлений и валидацию форм на основе реактивных Runes.

Модуль машинного обучения на Python/FastAPI включает 11 моделей: прогноз надоев на корову (XGBoost с квантильной регрессией q10/q50/q90,

обеспечивающей доверительные интервалы) и на стадо (Prophet с декомпозицией тренда и сезонности), обнаружение мастита и кетоза, выявление охоты, оценка риска выбраковки, оценка BCS, кластеризация стада (KMeans с автоматическим выбором  $k$  по силуэту), аномалии оборудования (Isolation Forest) и рекомендации по кормлению. MLOps-инфраструктура включает: автоматическое переобучение (APScheduler), оптимизацию гиперпараметров (Optuna, TPE-сэмплер, 8-мерное пространство), мониторинг дрейфа данных (prediction drift и feature drift), версионирование моделей и отслеживание экспериментов (MLflow).

В серверной части реализованы алгоритмы предиктивной аналитики на Rust: метод Хольта–Винтерса для прогнозирования временных рядов надоя с обнаружением выбросов (IQR) и структурных сдвигов (t-критерий); кривая лактации Вуда для моделирования зависимости надоя от DIM; комплексный индекс здоровья на основе Z-оценок четырёх показателей (надой, жвачка, активность, SCC) с динамической визуализацией тренда за 90 дней; оптимизатор запуска; система сравнения 8 моделей прогнозирования (SES, SMA, WMA, линейная регрессия, метод Хольта, Хольт–Винтерс, ряд Фурье, ARIMA(0,1,1)) с автоматическим выбором лучшей по MAPE. Реализован ансамблевый прогноз, объединяющий ML-модель (XGBoost) и лучшую Rust-модель методом взвешенного среднего с весами, обратно пропорциональными MAPE, что повышает робастность прогнозирования. Для обеспечения интерпретируемости прогнозов внедрён метод SHAP, визуализирующий вклад каждого признака в виде горизонтальной столбчатой диаграммы в интерфейсе.

Основные количественные результаты разработки включают: серверную часть из 25 модулей обработчиков, 33 модулей сервисов, 24 модулей моделей данных и 6 модулей middleware; базу данных из более 35 таблиц, более 20 файлов миграций с расширениями TimescaleDB и pg\_trgm; клиентское приложение из более 24 страниц, 23 модулей API-клиента и более 16 UI-компонентов; модуль машинного обучения из 11 моделей с автоматическим обучением и переобучением, мониторингом дрейфа и оптимизацией гиперпараметров через Optuna; серверную аналитику из 8 моделей прогнозирования с автоматическим сравнением по MAPE/RMSE, индексом здоровья и кривой Вуда; интеграцию с Lely, охватывающую 23 типа синхронизируемых данных с шифрованием AES-256-GCM и

инкрементальной синхронизацией; мониторинг через стек Grafana LGTM для сбора метрик, логов и трассировок; тестирование из 30+ тестовых файлов с нагрузочным тестированием через k6; и конвейер CI/CD с параллельным тестированием трёх компонентов и автоматическим деплоем при создании тегов.

Научная новизна работы заключается в архитектурном решении, позволяющем серверным алгоритмам прогнозирования на Rust работать автономно, обеспечивая предиктивную аналитику даже при недоступности ML-сервиса, а также в реализации системы автоматического сравнения 8 моделей прогнозирования с выбором оптимальной для каждого животного индивидуально и ансамблевого объединения прогнозов ML и Rust-моделей с весами, обратно пропорциональными MAPE.

Практическая значимость подтверждается полнотой покрытия бизнес-процессов молочной фермы: от учёта животных и надоев до предиктивной аналитики и интеграции с роботизированным оборудованием. Система развёрнута в контейнерах Docker с обратным прокси Nginx, SSL-терминацией и мониторингом, что обеспечивает готовность к промышленной эксплуатации.

Дальнейшее развитие системы может включать: расширение набора моделей машинного обучения (прогнозирование заболеваемости конечностей, оптимизация параметров доильных роботов); интеграцию с дополнительными производителями оборудования (DeLaval, GEA); разработку мобильного приложения для фермеров с push-уведомлениями; добавление модулей финансового анализа и планирования с учётом стоимости кормов, ветеринарных услуг и рыночных цен на молоко; внедрение online learning для адаптации моделей к новым данным в реальном времени.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Berckmans D. General Introduction to Precision Livestock Farming // Animal Feed Science and Technology. — 2017. — Т. 228. — С. 4—9.
2. Bewley J. M. Precision Dairy Farming: Advanced Analysis Solutions for the Modern Dairy Farm // Pennsylvania State University Extension. — 2016.
3. The Rust Programming Language. — URL: <https://www.rust-lang.org/> (дата обращения 20.04.2026).
4. Axum: Ergonomic and Modular Web Framework Built with Tokio, Tower, and Hyper. — URL: <https://github.com/tokio-rs/axum> (дата обращения 20.04.2026).
5. SQLx: The Rust SQL Toolkit. — URL: <https://github.com/launchbadge/sqlx> (дата обращения 20.04.2026).
6. PostgreSQL: The World's Most Advanced Open Source Database. — URL: <https://www.postgresql.org/> (дата обращения 20.04.2026).
7. TimescaleDB: The PostgreSQL Extension for Time-Series Data. — URL: <https://www.timescale.com/> (дата обращения 20.04.2026).
8. SvelteKit: Web Development, Streamlined. — URL: <https://kit.svelte.dev/> (дата обращения 20.04.2026).
9. Tailwind CSS: Rapidly Build Modern Websites Without Ever Leaving Your HTML. — URL: <https://tailwindcss.com/> (дата обращения 20.04.2026).
10. Chart.js: Open Source HTML5 Charts. — URL: <https://www.chartjs.org/> (дата обращения 20.04.2026).
11. XGBoost Documentation. — URL: <https://xgboost.readthedocs.io/> (дата обращения 20.04.2026).
12. Prophet: Automatic Forecasting Procedure. — URL: <https://facebook.github.io/prophet/> (дата обращения 20.04.2026).
13. FastAPI: Modern, Fast Web Framework for Building APIs with Python. — URL: <https://fastapi.tiangolo.com/> (дата обращения 20.04.2026).
14. Docker: Accelerated, Containerized Application Development. — URL: <https://www.docker.com/> (дата обращения 20.04.2026).

15. Docker Compose Overview. — URL: <https://docs.docker.com/compose/> (дата обращения 20.04.2026).
16. Nginx: High Performance Load Balancer, Web Server, and Reverse Proxy. — URL: <https://nginx.org/> (дата обращения 20.04.2026).
17. Prometheus: Monitoring System and Time Series Database. — URL: <https://prometheus.io/> (дата обращения 20.04.2026).
18. Grafana: The Open Observability Platform. — URL: <https://grafana.com/> (дата обращения 20.04.2026).
19. GitHub Actions: Automate Your Workflows. — URL: <https://docs.github.com/en/actions> (дата обращения 20.04.2026).
20. Sensors and Clinical Mastitis — The Quest for the Perfect Alert / Н. Hogeveen [и др.] // Journal of Dairy Science. — 2010. — Т. 93, № 5. — С. 1922—1930.
21. Tokio: A Runtime for Writing Reliable Asynchronous Applications with Rust. — URL: <https://tokio.rs/> (дата обращения 20.04.2026).
22. Tower: Modular and Reusable Components for Building Robust Networking Clients and Servers. — URL: <https://github.com/tower-rs/tower> (дата обращения 20.04.2026).
23. TimescaleDB Hypertables. — URL: <https://docs.timescale.com/use-timescale/latest/hypertables/> (дата обращения 20.04.2026).
24. Svelte 5: Runes. — URL: <https://svelte-5-preview.vercel.app/> (дата обращения 20.04.2026).
25. Optuna: Hyperparameter Optimization Framework. — URL: <https://optuna.org/> (дата обращения 20.04.2026).
26. Grafana Loki: Log Aggregation System. — URL: <https://grafana.com/docs/loki/> (дата обращения 20.04.2026).
27. Grafana Tempo: High-Volume Distributed Tracing Backend. — URL: <https://grafana.com/docs/tempo/> (дата обращения 20.04.2026).
28. NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM). — URL: <https://csrc.nist.gov/pubs/sp/800/38/d/final> (дата обращения 20.04.2026).

29. PostgreSQL pg\_trgm Extension. — URL: <https://www.postgresql.org/docs/current/pgtrgm.html> (дата обращения 20.04.2026).
30. utoipa: OpenAPI Documentation Generation for Rust. — URL: <https://github.com/juhaku/utoipa> (дата обращения 20.04.2026).
31. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519. — 05.2015. — URL: <https://tools.ietf.org/html/rfc7519> (дата обращения 20.04.2026).
32. OWASP Cross-Site Request Forgery Prevention Cheat Sheet. — URL: [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html) (дата обращения 20.04.2026).
33. Cross-Origin Resource Sharing: MDN Web Docs. — URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> (дата обращения 20.04.2026).
34. Scikit-learn: Clustering: K-Means. — URL: <https://scikit-learn.org/stable/modules/clustering.html> (дата обращения 20.04.2026).
35. Scikit-learn: Outlier Detection: Isolation Forest. — URL: [https://scikit-learn.org/stable/modules/outlier\\_detection.html](https://scikit-learn.org/stable/modules/outlier_detection.html) (дата обращения 20.04.2026).
36. SHAP: SHapley Additive exPlanations. — URL: <https://shap.readthedocs.io/> (дата обращения 20.04.2026).
37. MLflow: A Platform for the Machine Learning Lifecycle. — URL: <https://mlflow.org/> (дата обращения 20.04.2026).
38. Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. Holt-Winters' Seasonal Method. — URL: <https://otexts.com/fpp3/holt-winters.html> (дата обращения 20.04.2026).
39. Wood P. D. P. Factors Affecting the Shape of the Lactation Curve in Cattle // Animal Production. — 1967. — Т. 9, № 3. — С. 375—386.
40. GitHub Container Registry. — URL: <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry> (дата обращения 20.04.2026).
41. Vitest: Next Generation Testing Framework. — URL: <https://vitest.dev/> (дата обращения 20.04.2026).



42. Grafana k6: Load Testing for Engineering Teams. — URL: <https://k6.io/>  
(дата обращения 20.04.2026).

## ПРИЛОЖЕНИЕ А

### Исходный код

На рисунке А.1 изображён QR-код со ссылкой на GitHub репозиторий с исходным кодом разработанной информационной системы.



Рисунок А.1 – QR-код на репозиторий

## ПРИЛОЖЕНИЕ Б

### Описание таблиц базы данных

В данном приложении приведён перечень основных таблиц базы данных PostgreSQL с указанием назначения и ключевых полей.

В таблице Б.1 представлены таблицы учёта и воспроизводства.

Таблица Б.1 – Таблицы учёта животных и воспроизводства

| Таблица       | Назначение                                                     |
|---------------|----------------------------------------------------------------|
| users         | Пользователи: username, password_hash, role                    |
| animals       | Реестр животных: life_number, name, gender, birth_date, active |
| sires         | Быки-производители: name, code, breed                          |
| calvings      | Отёлы: animal_id, date, calf_id                                |
| calves        | Телята: calving_id, life_number, gender                        |
| inseminations | Осеменения: animal_id, date, sire_id                           |
| pregnancies   | Стельности: animal_id, date, expected_due_date                 |
| heats         | Охоты: animal_id, date, activity_level                         |
| dry_offs      | Запуски: animal_id, date                                       |
| transfers     | Переводы: animal_id, from_location, to_location                |
| bloodlines    | Кровные линии: animal_id, sire_id, dam_id                      |
| locations     | Местоположения: name, type, capacity                           |
| contacts      | Контакты: name, phone, role                                    |

В таблице Б.2 представлены таблицы надоев, кормления и мониторинга.

Таблица Б.2 – Таблицы надоев, кормления и мониторинга

| Таблица              | Назначение                                                  |
|----------------------|-------------------------------------------------------------|
| milk_day productions | Суточные надои: animal_id, date, milk_kg, fat, protein, scc |
| milk_visits          | Визиты на доение: animal_id, start_time, milk_kg, robot_id  |
| milk_quality         | Качество молока: animal_id, date, scc, fat, protein         |
| milk_visit_quality   | Качество по визитам: visit_id, scc, conductivity            |
| robot_milk_data      | Данные роботов: visit_id, quarter_times                     |
| feed_types           | Типы кормов: name, cost, unit                               |
| feed_groups          | Группы кормов: type_id, group_name                          |
| feed_day_amounts     | Дневные объёмы: animal_id, planned_kg, actual_kg            |
| feed_visits          | Визиты на кормление: animal_id, time, amount_kg             |
| activities           | Активность: animal_id, date, steps                          |
| ruminations          | Жвачка: animal_id, date, minutes                            |
| grazing_data         | Пастьба: animal_id, date, hours                             |
| bulk_tank_tests      | Резервуар: date, volume, temperature                        |

В таблице Б.3 представлены системные таблицы.

Таблица Б.3 – Системные таблицы

| Таблица                | Назначение                                         |
|------------------------|----------------------------------------------------|
| alerts                 | Оповещения: animal_id, category, severity, status  |
| tasks                  | Задачи: title, assignee, status, priority          |
| audit_log              | Аудит: user_id, action, entity, timestamp          |
| inventory_items        | Запасы: name, quantity, unit                       |
| inventory_transactions | Операции с запасами: item_id, type, quantity       |
| vet_records            | Ветеринария: animal_id, date, diagnosis            |
| lely_config            | Конфигурация Lely: base_url, encrypted_credentials |
| lely_sync_state        | Состояние синхронизации: entity_type, last_sync_at |
| system_settings        | Настройки: key, value                              |
| user_preferences       | Предпочтения: user_id, theme, page_size            |
| revoked_tokens         | Отозванные токены: jti, expires_at                 |
| weather_cache          | Кэш погоды: date, temperature                      |
| farm_zones             | Зоны фермы: name, type, boundaries                 |
| culling_events         | Выбраковки: animal_id, date, reason                |

## ПРИЛОЖЕНИЕ В

### Визуализация модели данных

В данном приложении представлена полная модель данных информационной системы в виде ER-диаграмм, разделённых по предметным областям. Диаграммы отображают все 35+ таблиц базы данных PostgreSQL, их ключевые столбцы и отношения между таблицами. Таблицы, отмеченные символом  $\Rightarrow$ , являются гипертаблицами TimescaleDB с автоматической партицировкой по времени.

#### Пользователи и реестр животных

На рисунке В.1 представлены основные таблицы пользователей и реестра животных. Таблица `animals` является центральной сущностью системы, на которую ссылается большинство остальных таблиц через внешний ключ `animal_id`. Таблица `users` обеспечивает аутентификацию и авторизацию.

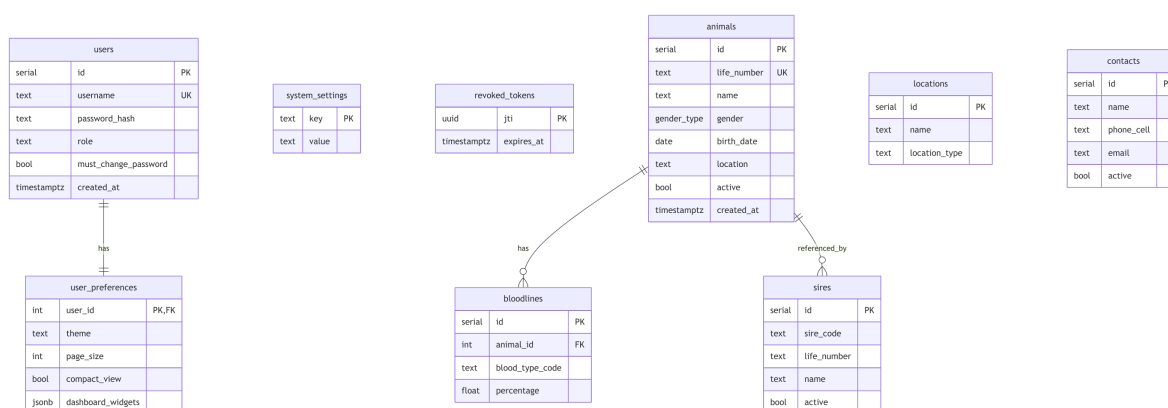


Рисунок В.1 – ER-диаграмма: пользователи и реестр животных

#### Воспроизводство и выбраковка

На рисунке В.2 представлены таблицы воспроизводства. Каждая таблица связана с животным через внешний ключ `animal_id`. Таблица `calves` связана с `calvings` каскадным удалением. Таблица `culling_events` используется как источник меток для ML-модели прогнозирования выбраковки.

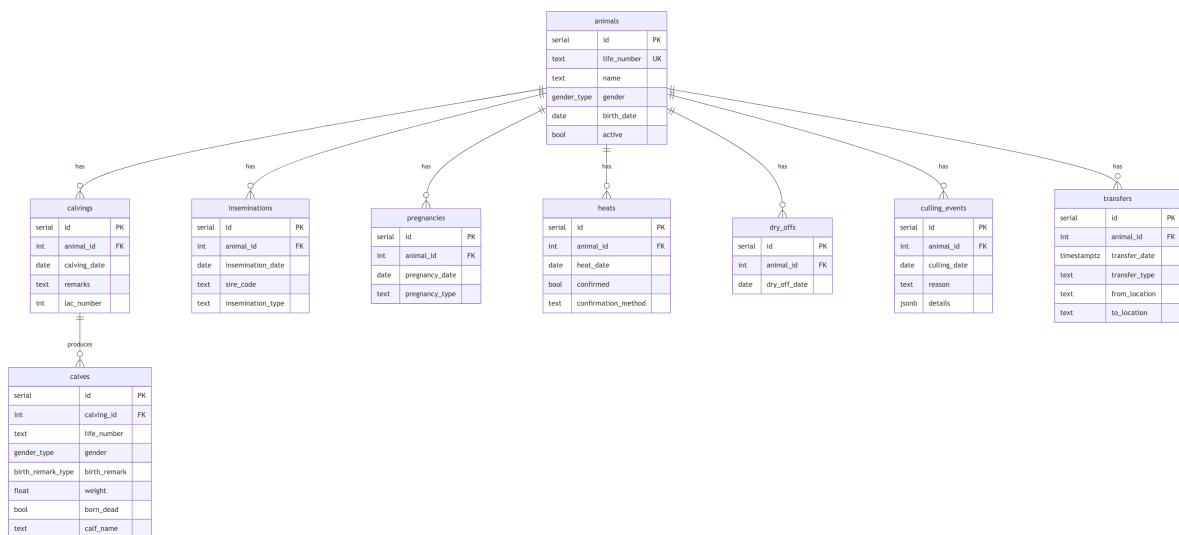


Рисунок В.2 – ER-диаграмма: воспроизводство и выбраковка

### Доеение и молоко

На рисунке В.3 представлены таблицы учёта надоев. Все таблицы, кроме `bulk_tank_tests`, привязаны к конкретному животному. Таблицы `milk_day productions`, `milk_visits`, `milk_quality`, `milk_visit_quality` и `robot_milk_data` являются гипертаблицами TimescaleDB с партицировкой по 7 дней.

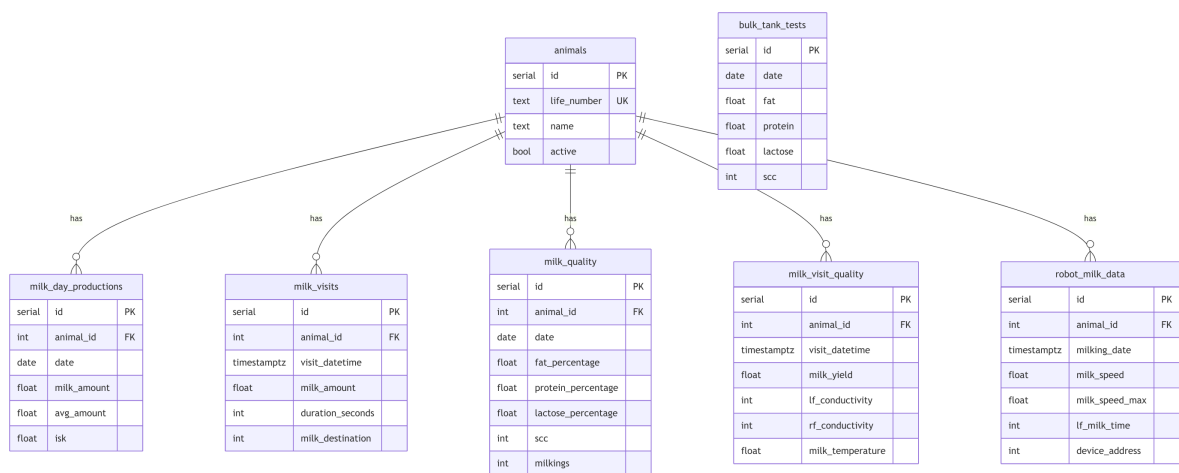


Рисунок В.3 – ER-диаграмма: доение и молоко

### Кормление, здоровье и активность

На рисунке В.4 представлены таблицы кормления и мониторинга здоровья. Таблицы `feed_day_amounts`, `feed_visits`, `activities` и `ruminations` являются гипертаблицами TimescaleDB. Таблица `vet_records` содержит поля `diagnosis_code` и `confirmed`,

используемые как метки для ML-моделей.

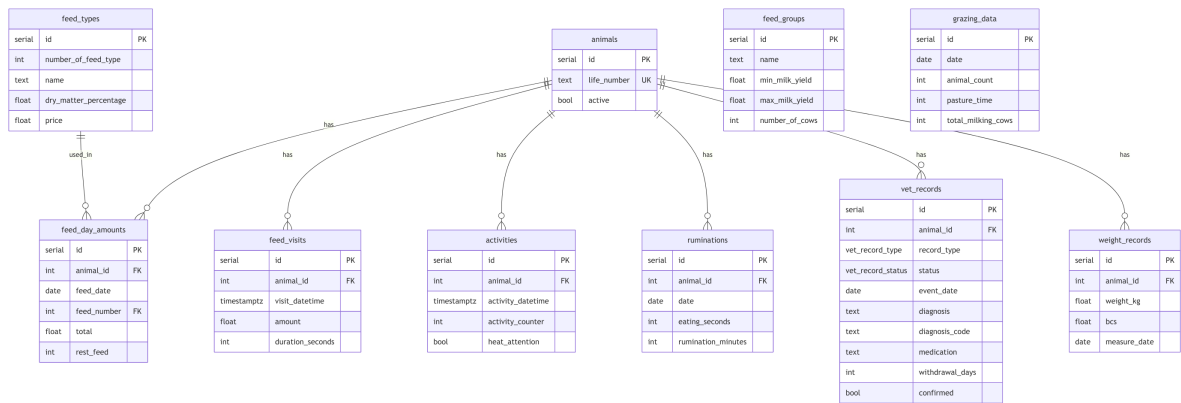


Рисунок В.4 – ER-диаграмма: кормление, здоровье и активность

**Оповещения и операции**

На рисунке В.5 представлены таблицы оповещений, задач, аудита, финансов и учёта запасов. Оповещения создаются автоматически движком правил `alert_engine` и доставляются через настраиваемые каналы уведомлений (браузер, Telegram).

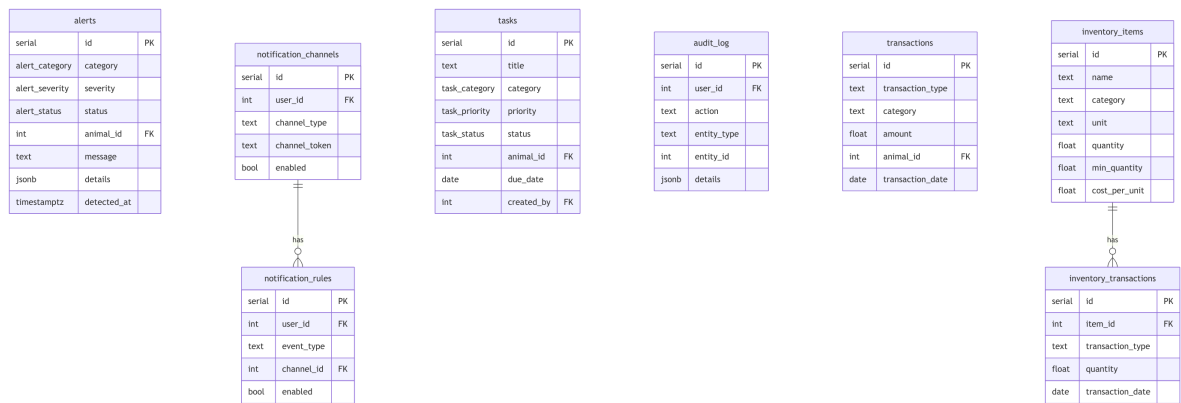


Рисунок В.5 – ER-диаграмма: оповещения и операции

**Системные таблицы**

На рисунке В.6 представлены системные таблицы: конфигурация интеграции с Lely с зашифрованными учётными данными, состояние синхронизации, зоны фермы и кэш погодных данных.



| lely_config |                    |    |
|-------------|--------------------|----|
| int         | id                 | PK |
| bool        | enabled            |    |
| text        | base_url           |    |
| text        | username           |    |
| text        | password_encrypted |    |
| text        | farm_key_encrypted |    |
| int         | sync_interval_secs |    |

| lely_sync_state |                |    |
|-----------------|----------------|----|
| serial          | id             | PK |
| text            | entity_type    | UK |
| timestampz      | last_synced_at |    |
| text            | status         |    |
| bigint          | records_synced |    |
| text            | error_message  |    |

| sync_log   |                |    |
|------------|----------------|----|
| serial     | id             | PK |
| text       | entity_type    |    |
| timestampz | last_synced_at |    |
| text       | status         |    |
| bigint     | records_synced |    |

| farm_zones |               |    |
|------------|---------------|----|
| serial     | id            | PK |
| text       | name          |    |
| text       | zone_type     |    |
| int        | capacity      |    |
| jsonb      | position_data |    |

| weather_cache |                  |    |
|---------------|------------------|----|
| date          | date             | PK |
| float         | temp_c           |    |
| float         | humidity         |    |
| float         | precipitation_mm |    |
| float         | wind_speed       |    |
| text          | weather_main     |    |
| timestampz    | fetches_at       |    |

Рисунок В.6 – ER-диаграмма: системные таблицы

## ПРИЛОЖЕНИЕ Г

### Примеры API-запросов и ответов

В данном приложении приведены примеры основных API-запросов и ответов информационной системы. Каждый рисунок содержит запрос и соответствующий ответ сервера.

```
1 POST /api/v1/auth/login
2 Content-Type: application/json
3
4 {
5     "username": "admin",
6     "password": "securepassword"
7 }
```

```
1 200 OK
2 Set-Cookie: token=eyJhbGci...; HttpOnly; SameSite=Strict
3 Set-Cookie: refresh_token=eyJhbGci...; HttpOnly; SameSite=
   Strict
4
5 {
6     "user": {
7         "username": "admin",
8         "role": "admin",
9         "must_change_password": false
10    }
11 }
```

Рисунок Г.1 – Аутентификация: POST /api/v1/auth/login

```
1 GET /api/v1/animals?page=1&page_size=25&gender=female
2 Cookie: token=eyJhbGci...
```

```
1 200 OK
2 {
3   "items": [
4     {
5       "id": 1,
6       "life_number": "RU123456789",
7       "name": "...",
8       "gender": "female",
9       "birth_date": "2020-03-15",
10      "active": true
11    }
12  ],
13  "total": 156,
14  "page": 1,
15  "page_size": 25
16 }
```

Рисунок Г.2 – Получение списка животных: GET  
/api/v1/animals

```
1 POST /api/v1/analytics/milk-trends
2 Content-Type: application/json
3
4 {
5   "animal_id": 1,
6   "days_ahead": 14
7 }
```

```
1 200 OK
2 {
3   "animal_id": 1,
4   "forecast": [
5     {
6       "date": "2026-04-21",
7       "predicted_milk_kg": 28.5,
8       "lower_bound": 25.1,
9       "upper_bound": 31.9
10    },
11    {
12      "date": "2026-04-22",
13      "predicted_milk_kg": 28.2,
14      "lower_bound": 24.8,
15      "upper_bound": 31.6
16    }
17  ],
18   "model": "holt_winters",
19   "last_actual_date": "2026-04-20"
20 }
```

Рисунок Г.3 – Прогноз надоев: POST  
/api/v1/analytics/milk-trends

```
1 POST /predict/mastitis
2 Content-Type: application/json
3 X-API-Key: <api_key>
4
5 {
6   "animal_id": 1,
7   "scc": 350000,
8   "conductivity": 8.5,
9   "fpr": 1.45,
10  "milk_change_pct": -15.0,
11  "dim": 45,
12  "lactation": 2
13 }
```

```
1 200 OK
2 {
3   "animal_id": 1,
4   "risk_level": "high",
5   "probability": 0.87,
6   "contributing_factors": [
7     {"factor": "scc_change", "importance": 0.35},
8     {"factor": "conductivity", "importance": 0.25},
9     {"factor": "milk_change", "importance": 0.20}
10  ]
11 }
```

Рисунок Г.4 – Обнаружение мастита: POST /predict/mastitis

```
1 GET /api/v1/reports/export/R16/csv?date_from=2026-04-01
2 Cookie: token=eyJhbGci...
```

```
1 200 OK
2 Content-Type: text/csv
3 Content-Disposition: attachment; filename="R16_2026-04-20.csv"
4
5 date,total_milk_kg,avg_milk_kg,avg_fat_pct
6 2026-04-01,4350.0,29.0,3.85
7 2026-04-02,4280.5,28.5,3.82
```

Рисунок Г.5 – Экспорт отчёта в CSV: GET  
/api/v1/reports/export